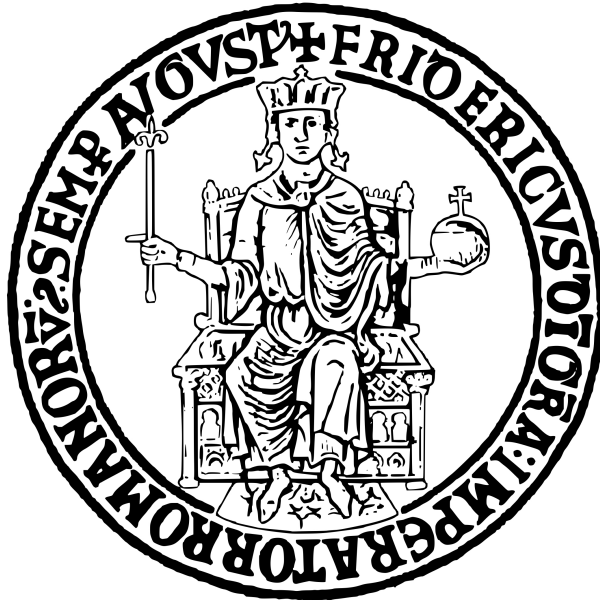


Università degli Studi di Napoli Federico II

Corso di Laurea in Informatica



DietiEstates25



Autori:

Giuliano Savino Matricola: N86004523

Anthony Arenella Matricola: N86004512

Giovanni Raucci Matricola: N86004635

Anno Accademico: 2024/2025

Indice

Introduzione	1
1 Struttura della Documentazione	1
Analisi dei requisiti	2
2 Ruoli del sistema	2
3 Requisiti individuati	2
3.1 Requisiti non funzionali di DietiEstates25	2
3.2 Requisiti di dominio per DietiEstates25	3
4 Modello dei casi d'uso	5
4.1 Tabella dei casi d'uso	5
4.2 Descrizione dettagliata dei casi d'uso	6
4.2.1 Utente	6
4.2.2 Cliente	6
4.2.3 Amministratore	6
4.2.4 Gestore agenzia immobiliare	7
4.2.5 Agente immobiliare	8
4.2.6 Utente non registrato	9
4.3 Use Case Diagram	10
5 Individuazione Target Utenti	12
5.1 Personas: Antonio esposito	12
5.2 Personas: Daniela Formicola	13
5.3 Personas: Luca esposito	14
5.4 Personas: Rosa bianchi	15
6 Prototipazione visuale tramite Mock-up	16
7 Descrizione testuali strutturate	21
7.1 Tabella di Cockburn del caso d'uso n.1 e relativi Mock-up	21
7.2 Tabella di Cockburn del caso d'uso n.2	23
8 Glossario	24
Documento di Design del Sistema	25

10 Architettura del Software	25
10.1 Client	25
10.2 Server	26
10.3 Struttura Schematizzata Back-End	27
10.4 Tecnologie Adottate	28
10.5 JavaFX	28
10.6 Jakarta Enterprise Edition	30
10.7 SceneBuilder	31
10.8 PostgreSQL	32
10.9 Docker	33
10.10 JavaScript	34
10.10.1 Perché JavaScript?	34
10.11 Altri servizi cloud usati	35
11 Motivazione delle scelte di design	36
11.1 Intro	36
11.2 Colori del software	36
11.3 Pagina di benvenuto	37
11.4 Analisi del design	37
11.5 Pagine di accesso	38
11.6 Aree riservate	39
11.7 Crea annuncio	39
11.8 Visualizza appuntamenti	40
11.9 Modifica profilo	41
11.10 Creazione account	42
11.11 Visualizza notifiche	43
11.12 Home di DietiEstates25	44
11.13 Ricerche passate	44
12 Diagrammi delle classi di design	46
12.1 Gestione delle notifiche	46
12.2 Effettua prenotazione	47
12.3 Registrazione	48
12.4 Cambio password	49
12.5 Crea annuncio	50
12.6 Modifica profilo	51
12.7 Autenticazione	52
12.8 Ricerca	53
12.9 Creazione account	54
12.10 Lascia recensione	55

13 Diagramma di sequenza di design	56
13.1 Caso d'uso : Prenota visita	56
13.2 Caso d'uso: Lascia recensione	58
 Artefatti Software	 59
14 DockerFile e file di build Automatica	59
14.1 File di build Automatica	59
14.2 DockerFile	59
14.2.1 Struttura del Dockerfile	59
14.2.2 Docker compose	60
14.2.3 6.1.3 Accesso Interattivo al Container	60
14.2.4 Gestione delle Password	61
 15 Strumento di versioning	 61
15.1 Scelte adottate	61
15.2 Repository	62
 16 Report di qualità del codice	 63
16.1 SonarQube	63
 Testing	 64
16.2 Unit Testing	64
16.2.1 lasciaRecensione	64
16.2.2 checkVisitaPerCliente	67
 17 Usabilità sul campo	 70
17.1 Raccolta feedback	70
 18 Valutazione degli esperti	 74
18.1 Fornire un feedback informativo	79
18.2 Prevenire gli errori	79
18.3 Consentire annullamenti semplici delle azioni	80
18.4 Ridurre il carico cognitivo	80
18.5 Test con utenti reali	80
18.5.1 Partecipanti	80
18.5.2 Funzionalità testate	81
18.5.3 Metriche considerate	81
18.5.4 Pilota il test	83
18.5.5 Risoluzione errori	83
18.5.6 Definizione Survey e Discussione	84

Introduzione

DietiEstates25 è un sistema software progettato con l'obiettivo di **semplificare la gestione di servizi immobiliari**. La piattaforma offre un'interfaccia intuitiva e comoda per semplificare la ricerca e l'interazione con annunci immobiliari, al fine di offrire all'utente un utilizzo rapido e piacevole.

1 Struttura della Documentazione

Il seguente documento è suddiviso in tre sezioni:

- **Documento dei requisiti software:** In questa sezione si analizzano a fondo le richieste fornite dal cliente elaborando con precisione tutti i **requisiti richiesti per la realizzazione del software**; Questa sezione rappresenta la prima parte fondamentale per la realizzazione del prodotto.
- **Documento di Design del Sistema:** In questa sezione è presente una **descrizione e motivazione delle tecnologie adottate per la realizzazione del sistema**.
- **Testing e Valutazione dell'usabilità:** In questa sezione della documentazione saranno illustrate le strategie di testing adottate per l'applicativo, **insieme alle tecniche di usabilità implementate per garantirne un'esperienza d'uso efficace e intuitiva**.

Analisi dei Requisiti

Per garantire la realizzazione di **DietiEstates25**, è fondamentale comprendere a fondo i requisiti del sistema. L'analisi dei requisiti verterà su:

- Identificazione degli attori principali del sistema
- Definizione dettagliata delle funzionalità chiave degli attori
- Definizione del target degli utenti
- Prototipazione dell'applicazione

2 Ruoli del sistema

A seguito di un'attenta analisi delle esigenze del committente, sono stati individuati i seguenti ruoli all'interno del sistema:

- **Amministratore: Ruolo di livello superiore nel sistema, responsabile della gestione generale dell'agenzia immobiliare.** L'amministratore accede al sistema utilizzando credenziali predefinite e ha la possibilità di creare account per altri utenti, tra cui account di supporto.
- **Gestore dell'agenzia immobiliare: Ruolo subordinato all'amministratore.** L'account del gestore viene creato dall'amministratore e, a sua volta, il gestore ha la facoltà di creare account di supporto per altri utenti.
- **Agente immobiliare: Rappresenta gli agenti immobiliari operanti nell'agenzia,** con compiti legati alla gestione delle proprietà e alla comunicazione con i clienti.
- **Cliente: Individui o entità che usufruiscono dei servizi offerti dagli agenti immobiliari,** interessati alla ricerca e alla prenotazioni di visiste.

3 Requisiti individuati

3.1 Requisiti non funzionali di DietiEstates25

- Il sistema deve essere sviluppato utilizzando un linguaggio **Object-Oriented**.
- Il sistema deve offrire un'interfaccia intuitiva e facile da utilizzare, con una **chiara visualizzazione delle informazioni relative agli immobili**.
- Il sistema deve garantire una ricerca rapida ed efficiente. In particolare, il tempo di risposta della ricerca deve essere inferiore a **2 secondi nel 99% delle richieste**, in modo da assicurare un requisito testabile per il futuro.
- Il sistema deve conservare le credenziali degli utenti in **modo sicuro**.

- Il sistema deve essere semplice da utilizzare. In particolare:
 - I clienti che utilizzano l'applicazione devono poter usufruire di tutte le funzionalità principali (login, prenotazione visita, visualizzazione notifiche, ecc.) in modo immediato, senza necessità di formazione.
 - Gli agenti immobiliari e i gestori devono poter acquisire familiarità con tutte le funzionalità del sistema entro un massimo di **30 minuti di formazione**. Dopo tale periodo, il numero di errori commessi non dovrebbe superare 1 all'ora durante l'utilizzo del sistema.
 - Nessun utente dovrebbe avere la necessità di richiedere supporto per l'uso ordinario del sistema.
- Il sistema deve garantire un tempo di disponibilità del **99% su base mensile**.

3.2 Requisiti di dominio per DietiEstates25

- Il sistema, in fase di creazione dell'annuncio, deve verificare che l'utente abbia inserito la classe energetica in conformità con il **Decreto Legislativo 192/2005**.
- Solo gli **agenti immobiliari possono caricare annunci di immobili**; tutti gli altri utenti non hanno questa possibilità.
- Gli amministratori, i gestori e gli agenti **non possono prenotare visite agli immobili**.
- Gli amministratori e i gestori **non possono lasciare recensioni agli agenti immobiliari**.
- Gli agenti immobiliari **non possono lasciare recensioni ad altri agenti né a sé stessi**.
- Gli immobili devono essere classificati in categorie come **appartamenti, ville, uffici e locali commerciali**.
- Il sistema deve garantire che ogni annuncio immobiliare contenga informazioni obbligatorie secondo la normativa vigente, tra cui **classe energetica, metratura, numero di stanze e prezzo**.

4 Modello dei casi d'uso

4.1 Tabella dei casi d'uso

Attore	Casi d'uso
Utente	• Accede al sistema • Ricerca immobili • Modifica profilo personale • Visualizza notifiche • Accede tramite credenziali di terze parti
Cliente	• Lascia recensione • Prenota visita
Amministratore	• Crea account supporto
Gestore azienda immobiliare	• Crea account per gli agenti immobiliari
Agente immobiliare	• Carica immobile • Accetta visita • Rifiuta visita • Visualizza appuntamenti • Aggiunge foto profilo
Utente non registrato	• Registra al sistema • Registra al sistema tramite credenziali di terze parti

4.2 Descrizione dettagliata dei casi d'uso

4.2.1 Utente

- **Accede al sistema:** L'utente inserisce email e password e accede al sistema.
- **Accede tramite google:** L'utente può cliccare *"Continua con google"* e accedere con le sue credenziali
- **Ricerca immobili tramite parametri:** L'utente **può cercare immobili filtrando per criteri come posizione, prezzo, numero di stanze e tipologia di immobile**. Il sistema restituisce un elenco di risultati corrispondenti con dettagli degli immobili cercati e immagini corrispondenti.
- **Modifica dati:** L'utente andando **nella sezione "Modifica profilo"** **può visualizzare i propri dati e modificarli**. Può modificare informazioni relativi alla sua email, password ma anche nome, cognome e numero di telefono.
- **Visualizza notifiche:** L'utente andando nella sezione "Notifiche" può visualizzare le sue notifiche e opzionalmente può anche decidere di **attivare o disattivare singole categorie di notifiche**. In base a che ruolo ricoprirà l'utente si andranno a visualizzare diverse categorie di notifiche.
- **Accede tramite google:** L'utente può **accedere al sistema tramite google**. Una volta effettuato l'accesso se questo ha avuto successo, l'utente accede al sistema.

4.2.2 Cliente

- **Prenota visita:** Il cliente può cliccare sull'immobile interessato, una **volta cliccato gli vengono visualizzate tutte le informazioni sull'immobile e può cliccare su "Prenota visita"**. Seleziona una data, seleziona una fascia oraria e clicca prenota
- **Lascia recensione:** Il cliente può cliccare sull'immobile interessato, una volta cliccato gli vengono visualizzate tutte le informazioni sull'immobile, insieme alle informazioni sull'immobile vi è anche un'area dedicata all'agente che ha caricato l'immobile. L'utente clicca "Lascia recensione", **seleziona il numero di stelle che vuole dare all'agente e clicca "Invia"**.

4.2.3 Amministratore

- **Creazione di account di supporto:** L'amministratore accede al sistema e seleziona l'opzione "Crea gestori". Una volta avviata la procedura, gli verrà presentato un modulo di registrazione, nel quale dovrà inserire i dati anagrafici del gestore da creare. Le informazioni richieste sono le seguenti:
 - Nome
 - Cognome
 - Codice Fiscale

- Numero di telefono
- Email
- Password

La decisione di far inserire i **dati anagrafici all'amministratore nella fase di creazione dell'account** è **motivata dalla necessità di garantire un'organizzazione e una gestione efficace del sistema**. In qualità di responsabile della gestione del personale dell'agenzia immobiliare, l'amministratore è l'unico autorizzato alla creazione degli account per i gestori. Essendo una figura centrale nell'organizzazione, deve avere accesso completo alle informazioni anagrafiche dei dipendenti sin dall'inizio del processo. Questo approccio garantisce una gestione centralizzata e sicura degli utenti, riduce il rischio di errori nella registrazione e ottimizza i flussi operativi, permettendo l'attivazione immediata degli account al primo accesso dei gestori.

4.2.4 Gestore agenzia immobiliare

- **Creazione di account per gli agenti immobiliari:** Il gestore accede al sistema e seleziona l'opzione "Crea agenti". Una volta avviata la procedura, gli verrà presentato un modulo di registrazione, nel quale dovrà inserire i dati anagrafici dell'agente da creare. Le informazioni richieste sono le seguenti:

- Nome
- Cognome
- Codice Fiscale
- Numero di telefono
- Email
- Password

Abbiamo adottato lo stesso principio seguito nella fase di creazione degli account per i gestori. Poiché il gestore è responsabile della gestione diretta degli agenti immobiliari, **è essenziale che disponga di tutte le informazioni anagrafiche necessarie per garantire una gestione strutturata ed efficace del personale**. Questo metodo assicura un controllo centralizzato, riduce la possibilità di errori e favorisce un accesso **immediato e sicuro al sistema per gli agenti**.

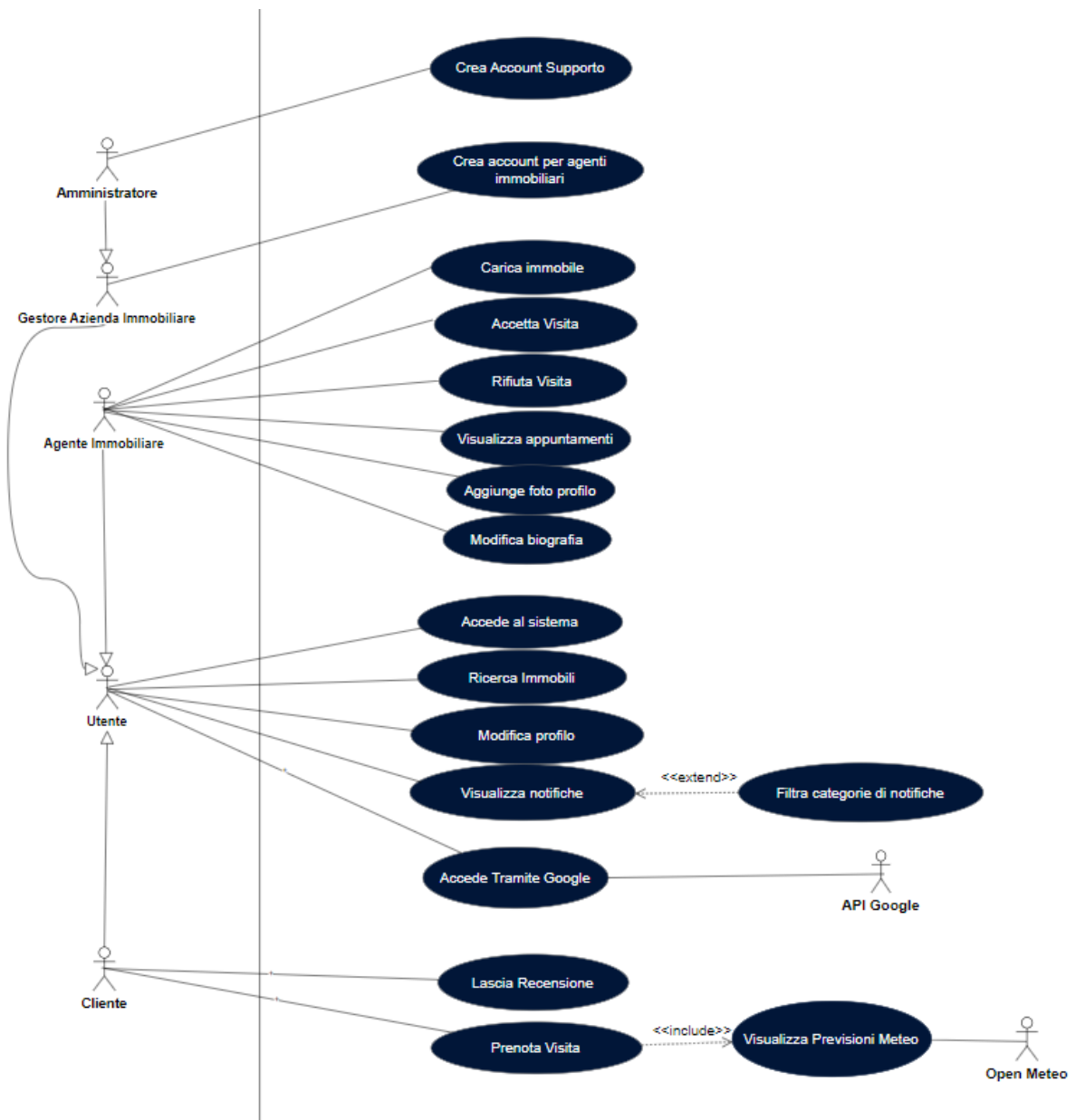
4.2.5 Agente immobiliare

- **Caricamento di un immobile:** L'agente accede al sistema e seleziona l'opzione "*Carica immobile*". Durante questa operazione, è tenuto a inserire una serie di informazioni dettagliate relative all'immobile, tra cui:
 - **Tipologia:** specifica se si tratta di un appartamento, una villa, un ufficio, ecc.
 - **Tipo di vendita:** indica se l'immobile è in vendita, in affitto.
 - **Posizione:** comprende l'indirizzo e la localizzazione geografica dell'immobile.
 - **Dimensioni:** superficie totale espressa in metri quadrati.
 - **Piano:** numero del piano in cui si trova l'immobile.
 - **Numero di locali:** numero totale di stanze presenti nell'immobile.
 - **Classe energetica:** certificazione dell'efficienza energetica dell'immobile.
 - **Prezzo:** valore richiesto per la vendita o l'affitto.
 - **Spese condominiali:** eventuali costi mensili per la gestione del condominio.
 - **Presenza di ascensore:** indica se l'immobile è dotato di ascensore.
 - **Fotografie:** immagini rappresentative dell'immobile per una migliore presentazione.
 - **Descrizione:** testo descrittivo che fornisce dettagli aggiuntivi sulle caratteristiche e i punti di forza dell'immobile.
- **Accettazione di una visita:** l'agente accede al sistema e seleziona la sezione "*Notifiche*", dove può visualizzare le richieste di visita ricevute. Dopo aver esaminato le notifiche, può confermare un appuntamento selezionando l'opzione "*Accetta visita*".
- **Rifiuto di una visita:** l'agente accede al sistema e seleziona la sezione "*Notifiche*", dove può visualizzare le richieste di visita ricevute. Dopo aver esaminato le notifiche, può confermare un appuntamento selezionando l'opzione "*Rifiuta visita*".
- **Visualizzazione degli appuntamenti:** l'agente accede al sistema e seleziona l'opzione "*Visualizza appuntamenti*". In questa sezione, può consultare l'elenco delle visite accettate e organizzare al meglio la propria agenda.
- **Modifica della foto profilo:** l'agente accede al sistema e seleziona l'opzione "*Modifica profilo*". Viene reindirizzato a una pagina in cui è visibile l'immagine attualmente impostata. Da questa sezione, può selezionare l'opzione "*Modifica foto*" e caricare una nuova immagine di profilo.
- **Modifica biografia :** L'agente accede al sistema e seleziona l'opzione "*Modifica profilo*". Viene reindirizzato a una pagina in cui è visibile la propria biografia attuale. Da questa sezione può selezionare l'opzione "*Modifica biografia*", l'agente carica la sua biografia il sistema fa una serie di controlli (come la lunghezza) e la biografia viene aggiornata.

4.2.6 Utente non registrato

- **Registrazione:** Un nuovo utente può creare un account fornendo nome,cognome,email e password. I dati inseriti sono soggetti a verifica da **parte del sistema** (ad esempio: **la email deve essere in un formato valido o la password deve essere abbastanza sicura**). Una volta effettuata la registrazione, se questa ha avuto successo, l'utente accede al sistema
- **Registrazione tramite google:**Un nuovo utente può cliccare "*Continua con google*" e accedere con le sue credenziali di terze parti.

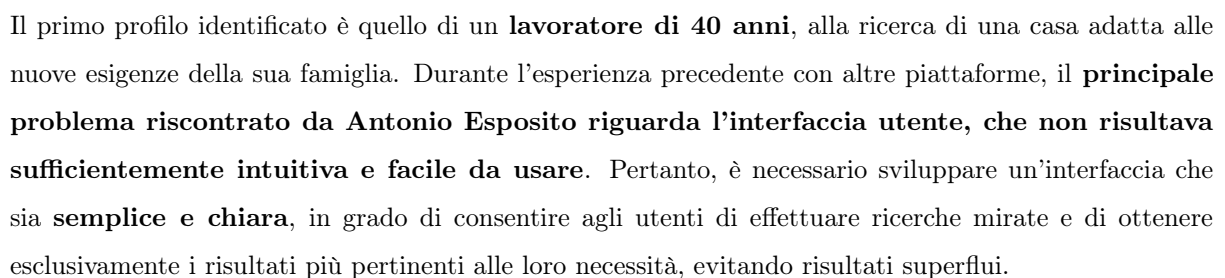
4.3 Use Case Diagram



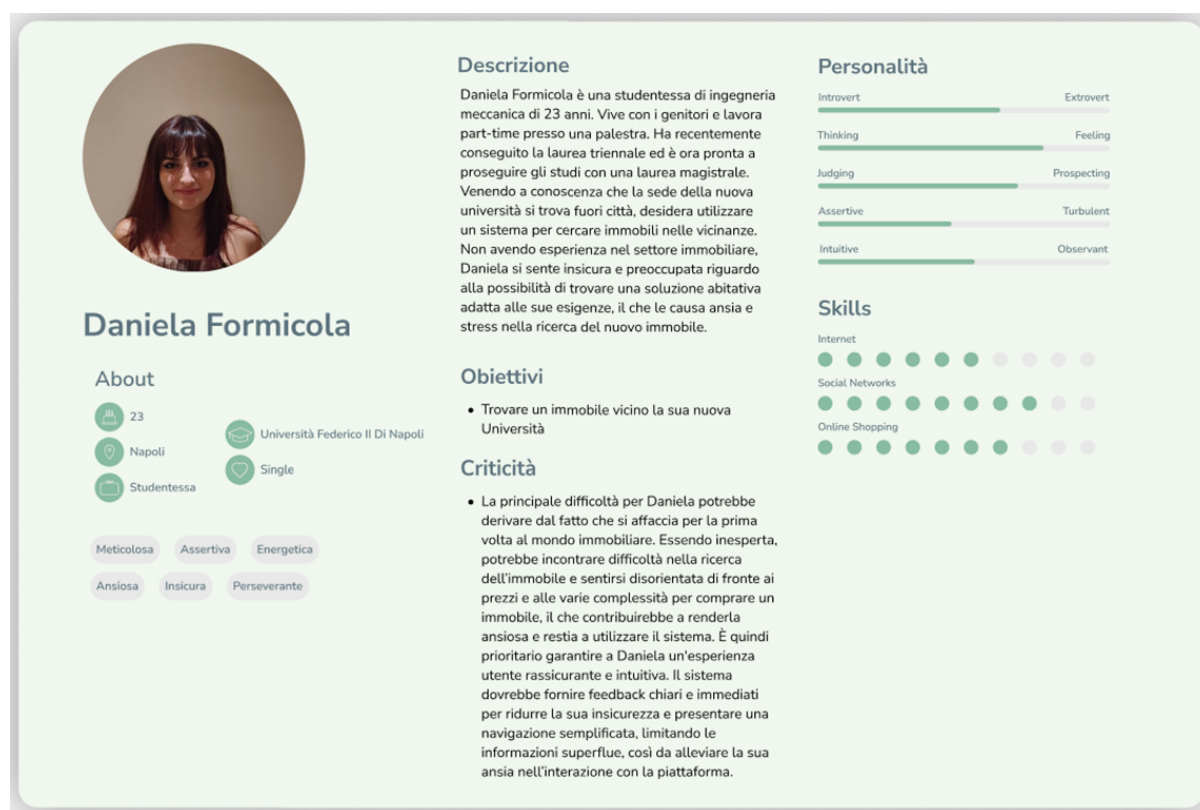


Durante la prima fase di sviluppo dell'intero software è cruciale il segmento di individuazione e caratterizzazione del Target degli utenti che utilizzeranno il software, in modo tale da soddisfare tutte le richieste e necessità presenti.

5.1 Personas: Antonio esposito



5.2 Personas: Daniela Formicola

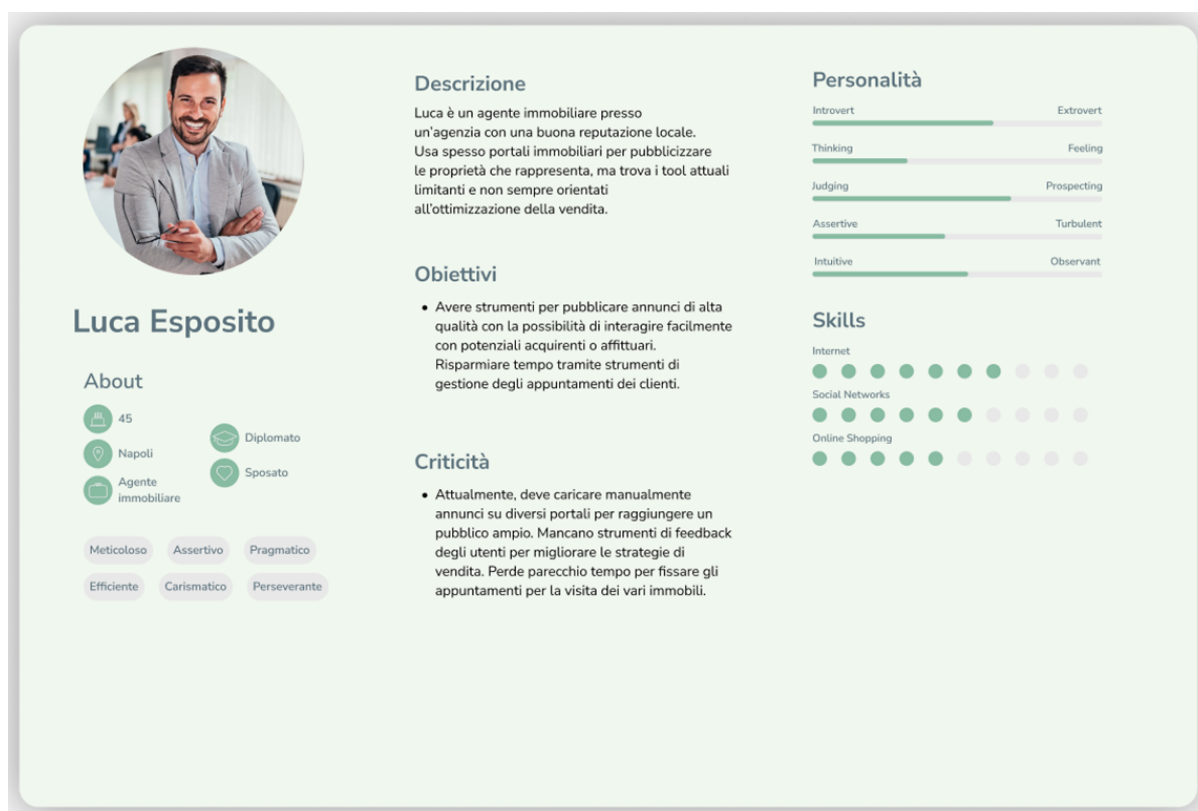


Il secondo profilo identificato è quello di Daniela Formicola, una **neolaureata triennale di 23 anni**, attualmente alla ricerca di una sistemazione nelle vicinanze della sua nuova sede universitaria. Il suo obiettivo è affrontare questo processo in modo semplice e privo di stress.

Dall'analisi della sua esperienza, è emerso come principale criticità la **scarsa familiarità con l'utilizzo delle piattaforme di ricerca immobiliare**, unita a una conoscenza limitata del **settore**, trattandosi della sua prima esperienza in questo ambito.

Per questo motivo, risulta essenziale progettare un'interfaccia intuitiva, chiara ed essenziale, priva di informazioni superflue che potrebbero generare confusione. Inoltre, è fondamentale che la piattaforma sia in grado di presentare le informazioni in modo semplice e accessibile, considerando che una parte degli utenti potrebbe trovarsi per la prima volta ad affrontare un processo di ricerca immobiliare.

5.3 Personas: Luca esposito

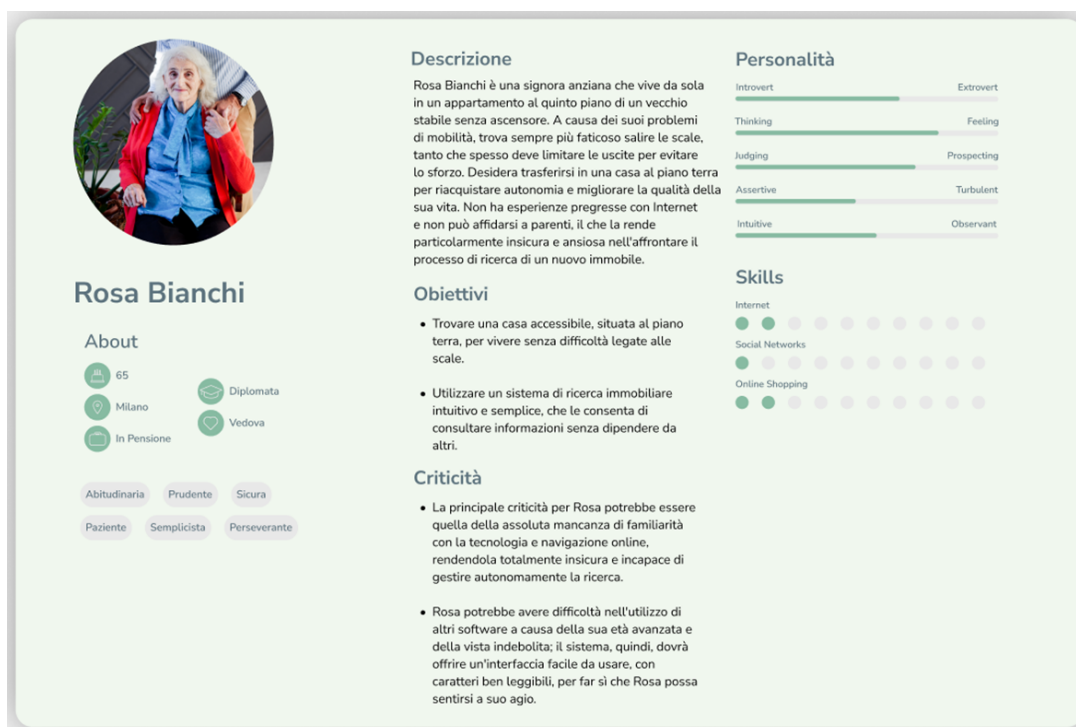


Il terzo profilo identificato è quello di Luca Esposito, un **agente immobiliare di 45 anni**, il cui interesse principale è rivolto alla velocità, all'ottimizzazione e alla qualità nel processo di gestione degli immobili.

Dall'analisi della sua esperienza, è emerso che Luca riscontra **difficoltà nell'utilizzo di altre piattaforme**, in particolare nella procedura di caricamento degli immobili, che spesso risulta complessa e poco intuitiva. Inoltre, le interfacce utente delle soluzioni attualmente disponibili presentano un eccesso di informazioni, rendendo l'interazione più onerosa.

Alla luce di queste considerazioni, il processo di caricamento degli immobili deve essere progettato in modo da risultare **semplice e immediato**, evitando la necessità per gli agenti immobiliari di consultare documentazione aggiuntiva o di affrontare complessità operative superflue.

5.4 Personas: Rosa bianchi



Il quarto profilo identificato è quello di Rosa Bianchi, una donna di 65 anni che vive da **sola ed è alla ricerca di un’abitazione accessibile e confortevole, in grado di rispondere alle sue specifiche esigenze.**

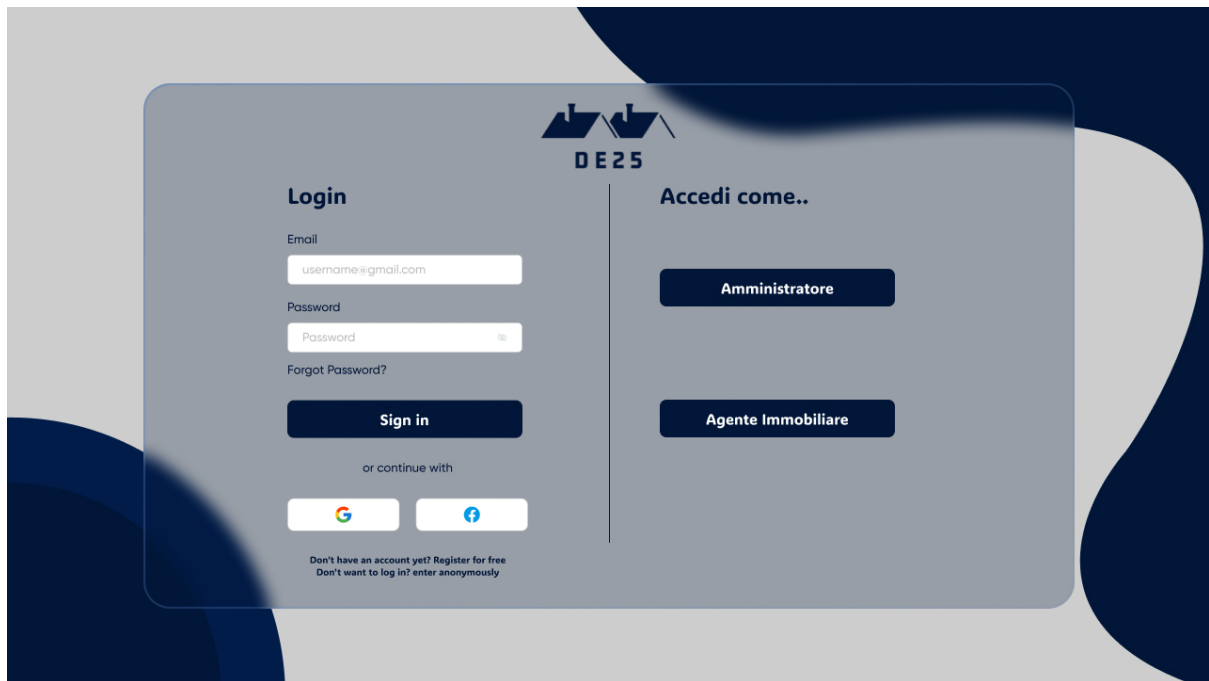
Dall’analisi della sua esperienza, è emersa una limitata **familiarità con l’uso delle piattaforme digitali**, evidenziando la necessità di sviluppare un’interfaccia intuitiva, chiara e facilmente leggibile. L’obiettivo è garantire una navigazione semplice e accessibile, anche per utenti con ridotte competenze informatiche o **difficoltà visive legate all’età**, facilitando il processo di ricerca dell’abitazione ideale senza ostacoli.

Alla luce di queste considerazioni, il sistema dovrà essere progettato in modo da presentare le informazioni in modo chiaro e leggibile, tenendo conto delle esigenze di utenti che potrebbero riscontrare difficoltà nella lettura e nell'interazione con le interfacce digitali.

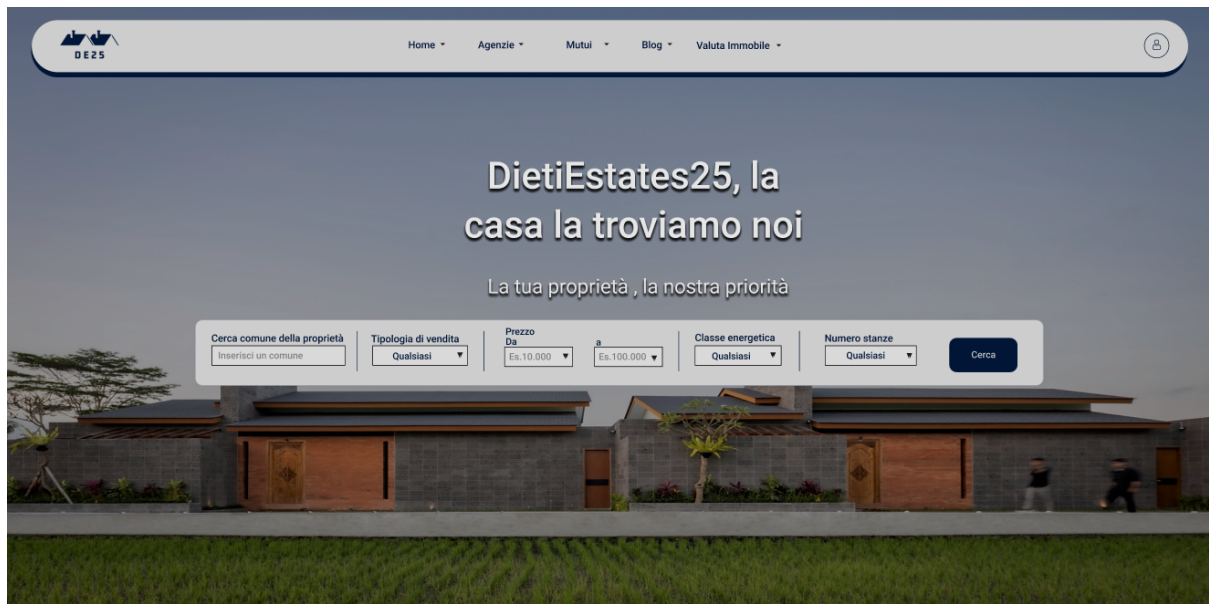
6 Prototipazione visuale tramite Mock-up

Prototipazione Visuale attraverso Mock-Up

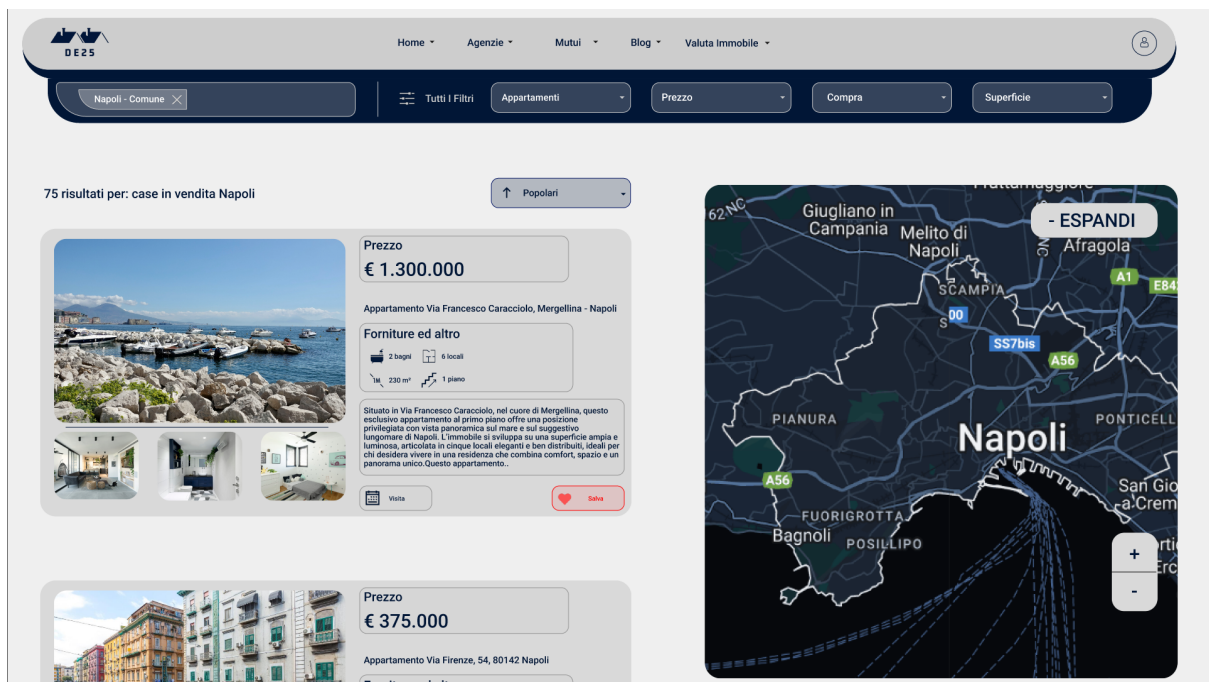
In questa sezione, verranno presentati i Mock-up realizzati per delineare il **flusso delle principali funzionalità di DietiEstates25**. Questi prototipi offrono una rappresentazione visiva delle schermate dell'applicazione e costituiscono uno strumento essenziale per raccogliere feedback preliminari sugli aspetti estetici e funzionali dell'interfaccia. **Grazie a questi modelli, è possibile analizzare e ottimizzare l'esperienza utente prima dello sviluppo definitivo.**



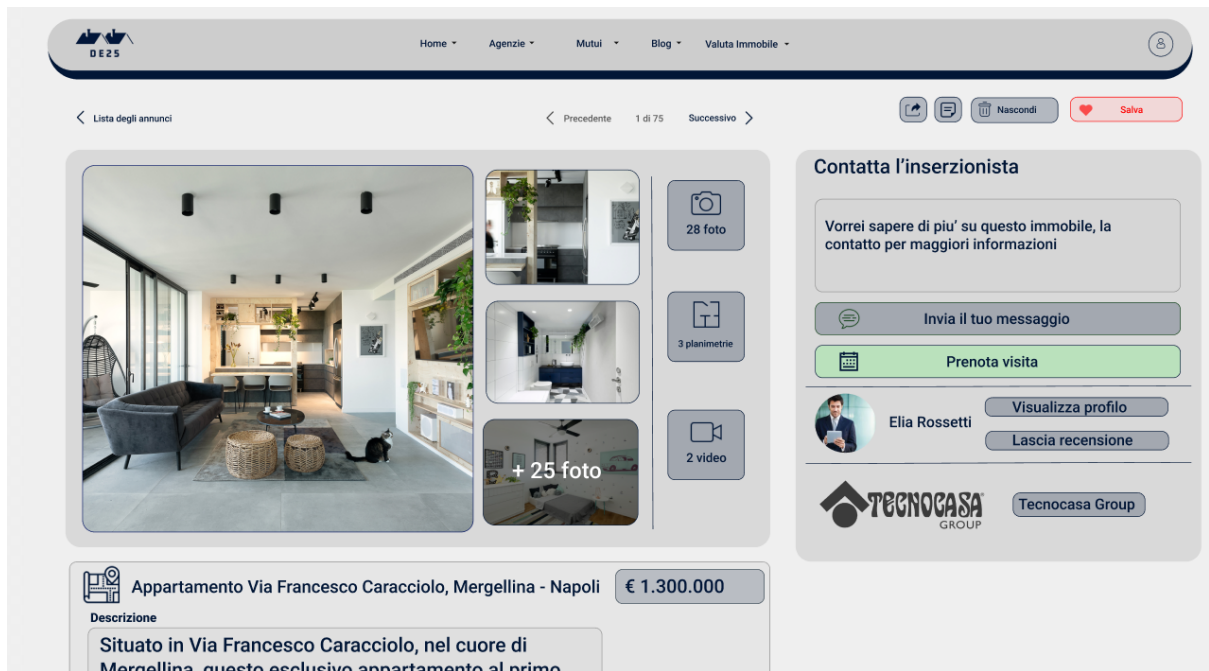
Mock-up M1 Pagina di benvenuto



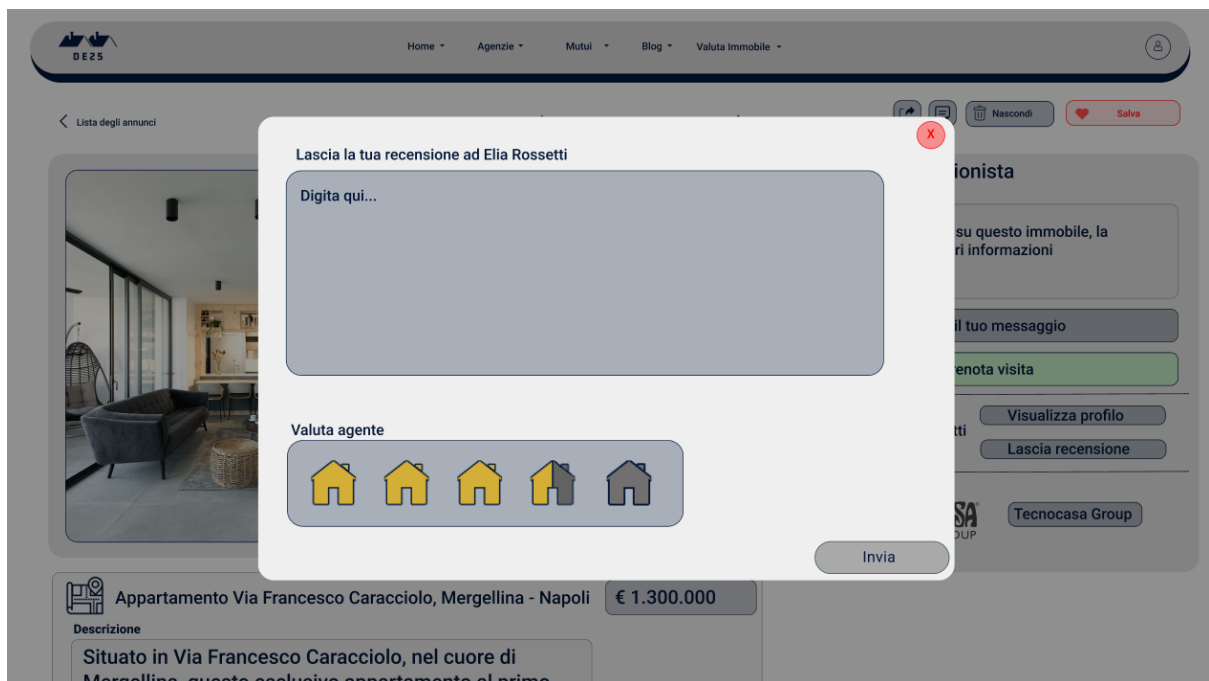
Mock-up M2 Home del sistema



Mock-up M3 Risultati ricerca

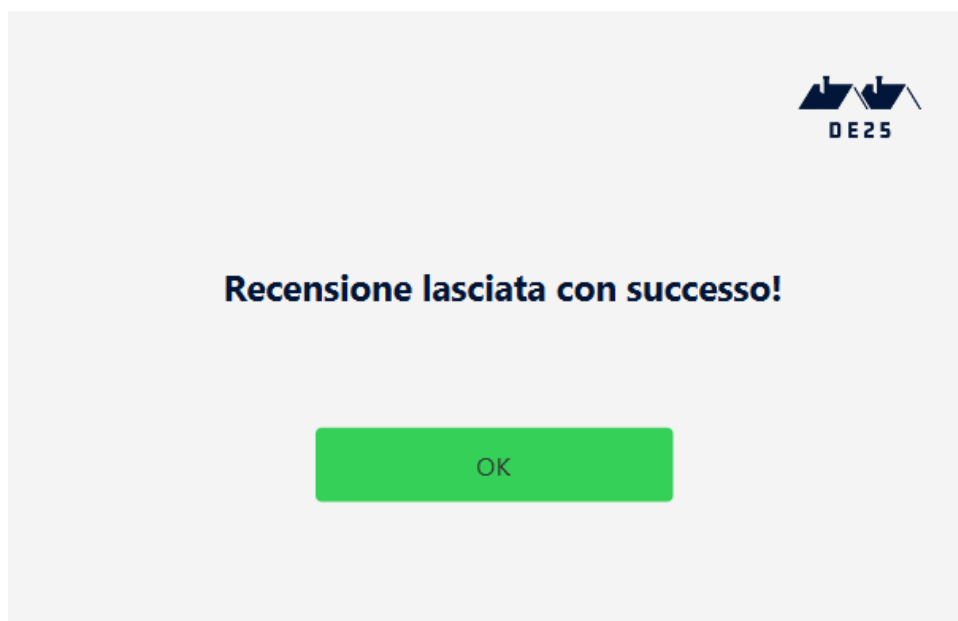


Mock-up M4 Visualizza immobile



Mock-up M5 Lascia recensione

Osservazione: Si noti che il pulsante "Invia" è disabilitato fin quando l'utente non digita un testo e inserisce una valutazione per l'agente.



Mock-up M6 Successo Lascia recensione



Mock-up M7 Prenota visita

Osservazione: Si noti che il pulsante "Prosegui" è disabilitato fin quando l'utente non seleziona un giorno per visita e una fascia oraria.



Prenotazione effettuata con successo!

OK

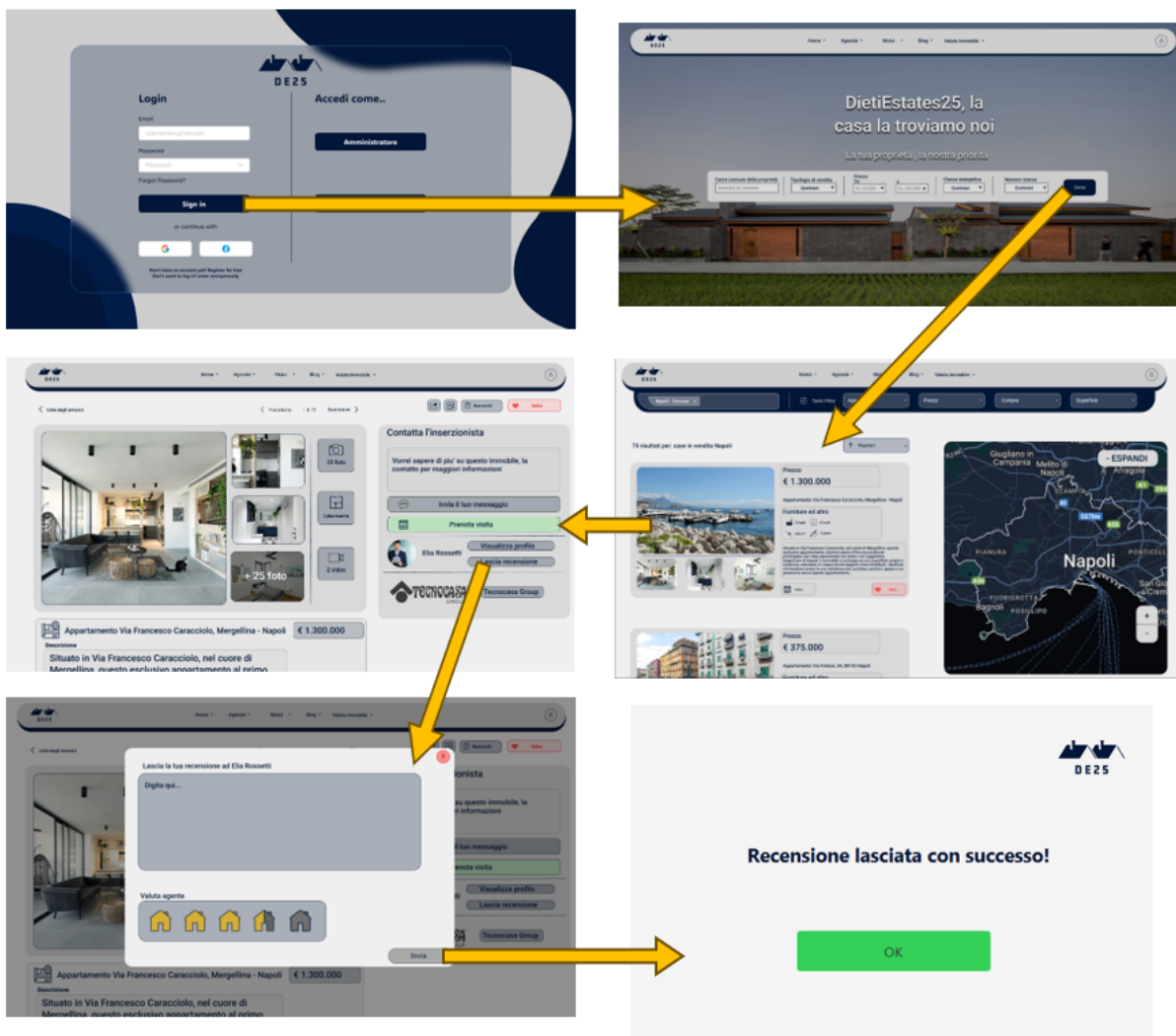
Mock-up M8 Successo Prenota visita

7 Descrizione testuali strutturate

In questa sezione verranno analizzati in dettaglio **alcuni dei principali casi d'uso identificati per il sistema**. L'analisi sarà condotta attraverso descrizioni testuali strutturate, utilizzando un formalismo tabellare basato sul modello di Cockburn, rappresentando in modo chiaro e sistematico l'interazione tra l'utente e il software, le condizioni di avvio e di successo, gli attori coinvolti e le eventuali eccezioni.

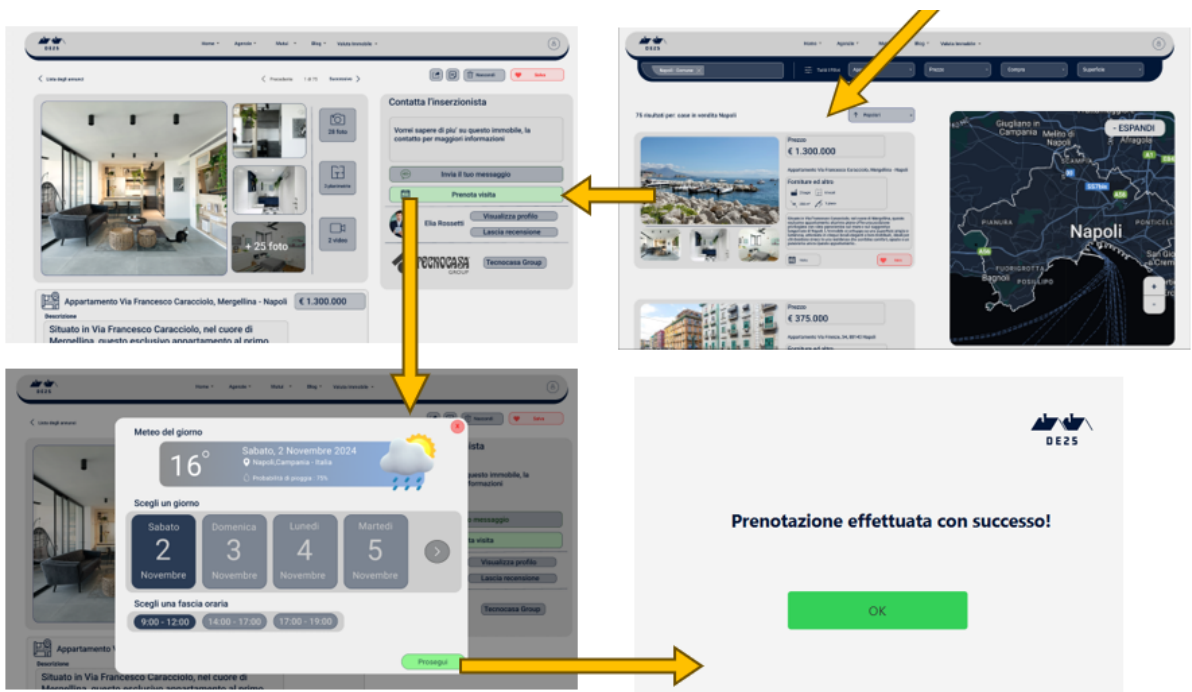
7.1 Tabella di Cockburn del caso d'uso n.1 e relativi Mock-up

USE CASE #1		Lascia Recensione Ad Agente	
Goal in Context	Il cliente potrà dare una valutazione sul servizio offerto dall'agente immobiliare		
Preconditions	L' utente deve aver effettuato l'accesso o deve essere registrato; L'utente deve aver effettuato una ricerca ed aver cliccato su un immobile.		
Success End Condition	Il Sistema tiene conto della recensione; recensione salvata; recensione appare sulla lista delle recensioni dell'agente		
Failed End Condition	Il Sistema non tiene conto della recensione		
Primary Actor	Cliente		
Trigger	Cliente clicca lascia recensione in M4 "Visualizza immobile"		
Main Scenario	Step n.	Cliente	System
	1	Clicca lascia recensione in M4 "Visualizza immobile"	
	2		Mostra M5 "Lascia recensione"
	3	Scrivo Recensione; Inserisce Numero Di Stelle	
	4	Clicca Invia Valutazione	
	5		Mostra M6 "Successo lascia recensione"
	6	Clicca "OK"	
	7		Mostra M4 "Visualizza immobile"
Extension Point A (Cliente clicca 'X' in M5 "Lascia recensione")	Step n.	Cliente	System
	1b	Clicca 'X'	
	2b		Mostra M4 "Visualizza immobile" e termina UC
Extension Point B (Cliente Inserisce Un Testo Non Valido)	Step n.	Cliente	System
	1c	Inserisce un testo non valido	
	2c	Seleziona numero di stelle	
	3c	Clicca invia valutazione	
	4c		Mostra Un Messaggio Di Errore a ridosso dei form e ritorno allo step 3 nel Main Scenario



7.2 Tabella di Cockburn del caso d'uso n.2

USE CASE #2	Prenota Visita		
Goal in Context	Il cliente vuole prenotare una visita di un immobile.		
Preconditions	L'utente deve aver effettuato l'accesso o deve essere registrato; Il cliente deve aver cercato un immobile; Il cliente deve cliccare su un immobile.		
Success End Condition	Il sistema dovrà tenere traccia della visita; Il sistema dovrà registrarla negli appuntamenti dell'agente; il sistema dovrà notificare l'agente.		
Failed End Condition	La prenotazione non viene effettuata; il sistema non tiene traccia della prenotazione; non arriva nessuna notifica all'agente.		
Primary Actor	Cliente		
Trigger	Il cliente clicca prenota una visita in M4 "Visualizza immobile"		
Main Scenario	Step n.	Cliente	System
	1	Clicca "Prenota visita" in M4 "Visualizza Immobile"	
	2		Mostra M7 "Prenota Visita"
	3	Seleziona il giorno	
	4	Seleziona la fascia oraria	
	5	Clicca "prosegui"	
	6		Mostra M8 "Messaggio di conferma"
	7	Clicca 'Ok'	
Extension Point A (Cliente "X" In M7 "Prenota visita")	Step n.	Cliente	System
	1a	Clicca "X" in M7 "Prenota visita"	
	2a		Ritorna a M4 "Visualizza Immobile" e termina UC



8 Glossario

- **UML (Unified Modeling Language)** : Un linguaggio di modellazione grafica standard utilizzato per visualizzare, specificare, costruire e documentare gli artefatti di un sistema software. Include vari tipi di diagrammi, come diagrammi delle classi, dei casi d'uso e delle sequenze.
- **Mockup** : Rappresentazione statica dell'interfaccia utente di un'applicazione, utilizzata per mostrare il layout visivo e simulare l'aspetto dell'interfaccia prima della realizzazione finale.
- **Casi d'uso** : Tecnica di analisi funzionale che descrive, in termini di attori e scenari, come il sistema interagisce con gli utenti per soddisfare determinati obiettivi. Ogni caso d'uso documenta le interazioni tra l'attore e il sistema.
- **Tabelle di cockburn** : Modello di rappresentazione dei casi d'uso proposto da Alistair Cockburn. Ogni tabella contiene informazioni come titolo, attore principale, precondizioni, flusso principale di eventi, estensioni e risultati attesi, rendendo i casi d'uso più strutturati e comprensibili.
- **Attore** : Entità esterna al sistema che interagisce con esso. Può essere un utente umano o un altro sistema che richiede servizi al software.
- **Personas** : sono rappresentazioni fittizie di utenti ideali, basate su dati reali, che aiutano a progettare prodotti, servizi o interfacce in modo più mirato.
- **Use Case Diagram** : Diagramma UML che rappresenta le interazioni tra gli attori e il sistema, mostrando le funzionalità principali (casi d'uso) offerte dal sistema
- **API(Application Programming Interface)** : Interfaccia che permette a diverse applicazioni di comunicare tra loro.
- **Sequence Diagram** : Diagramma UML che mostra come gli oggetti interagiscono in una sequenza temporale.
- **Front-end**: Parte di un'applicazione software che interagisce direttamente con l'utente.
- **Back-end** : Parte di un'applicazione software che gestisce la logica, i database e le operazioni di background.

Documento di Design del Sistema

In questa sezione vengono illustrate le scelte architetturali e i design pattern adottati nel software. Un design pattern è una soluzione generale e collaudata per risolvere problemi ricorrenti che si incontrano durante la progettazione di software. Questi pattern possono migliorare la comprensione del codice, la manutenzione e la scalabilità di un'applicazione

10 Architettura del Software

Il software sviluppato adotta un'architettura client-server, in cui il front-end è responsabile della presentazione dei dati e interagisce con il back-end tramite le API esposte. Il back-end fornisce i servizi e la logica di business, rendendo disponibili le API accessibili tramite rete, mantenendo così un'indipendenza strutturale rispetto alla parte front-end. La parte front-end è stata sviluppata utilizzando **JavaFX** e il server utilizzando **AWS**.

Sulla base delle specifiche appena esposte il sistema è organizzato in due sottosistemi principali:

- **Client:** Si occupa dell'interazione con l'utente, inviando richieste al *Server* ed elaborandone le risposte.
- **Server:** Gestisce le richieste pervenute dal *Client*, fa rispettare i vincoli esposti dalla business logic, interagisce con la base di dati relazionale per la persistenza dei dati e invia le risposte al *Client*.

10.1 Client

Il front-end è stato sviluppato utilizzando JavaFX. L'architettura del front-end segue un approccio ben strutturato e diviso in strati.

- **FXML (View):** Gli FXML definiscono l'**interfaccia utente** (UI) in modo dichiarativo, separando la logica di presentazione dalla logica di business. Ogni file FXML è collegato a un controller che gestisce gli eventi e le interazioni dell'utente.
- **Controller:** Questo livello gestisce le **interazioni con l'utente**, come i click sui pulsanti o l'inserimento dei dati. Non contiene logica di business, ma delega le operazioni a un ulteriore livello, il Facade.
- **Facade:** Questo strato funge da punto di accesso semplificato tra il front-end e il back-end. Il Facade incapsula la complessità delle chiamate al back-end e fornisce un'interfaccia **astratta ai controller**, i quali non devono preoccuparsi dei dettagli di comunicazione con il back-end. A sua volta, il Facade interagisce con il livello Service.
- **Service:** Questo livello si occupa della **comunicazione con il back-end**, effettuando le chiamate alle API esposte dal server e restituendo i dati elaborati al Facade.

Perché questa architettura?

La presente architettura può essere definita come una **Layered Architecture** arricchita dagli elementi dei design pattern **MVC** e **Facade**. Grazie alla netta separazione degli strati, ogni layer ha un ruolo ben definito, contribuendo a rendere il codice più modulare e facilmente manutenibile.

- **Separazione dei compiti:** In particolare, l'uso del layer **Facade** e del layer **Service** consente di isolare il Controller dalla logica di business. **Il Controller si occupa esclusivamente di gestire gli input dell'utente e di comunicare con il layer Facade**, il quale, a sua volta, interroga il **layer Service per eseguire la logica di business**. Questa separazione favorisce la scalabilità dell'architettura, rendendola pronta all'integrazione di nuove funzionalità senza richiedere modifiche ai componenti esistenti.
- **Miglior gestione degli errori:** Con questa architettura vi è anche una migliore gestione degli errori andandoli a centralizzare nel Facade che si occuperà di tradurre errori tecnici dovuti alle chiamate con le API esterne o semplicemente instradare dati verso il controller.
- **Facilità di adozione di nuove tecnologie:** Se decido di integrare nuove funzionalità, come il supporto a un'interfaccia web o mobile, potrei mantenere lo stesso Facade e Service, riducendo il lavoro necessario per l'adattamento.

10.2 Server

Il server è stato sviluppato utilizzando **Jakarta RESTful Web Services**, un framework Java che fornisce un'API per la creazione di servizi web RESTful. REST (*Representational State Transfer*) è uno stile architetturale per la progettazione di servizi web basato sui principi fondamentali della trasmissione dei dati tramite il protocollo HTTP.

L'architettura del server segue un pattern ben strutturato basato su una variante della **Three-Tier Architecture** e sulla separazione delle responsabilità. In particolare, si distinguono tre livelli principali:

- **Presentation Layer (Controller):** rappresenta il punto di ingresso dell'applicazione. Ha il compito di ricevere le richieste HTTP, validare i dati in input, delegare l'elaborazione al **Service Layer** e restituire una risposta HTTP appropriata.
- **Business Logic Layer (Service):** costituisce il cuore dell'applicazione. Si occupa della logica di business, interagisce con il **DAO** e con altri servizi, gestendo eventuali eccezioni o errori.
- **Data Access Layer (DAO):** è il livello responsabile della persistenza dei dati. Si occupa dell'interazione con il database ed esegue le operazioni **CRUD** (Create, Read, Update, Delete).

Perché questa architettura?

La seguente architettura può essere definita come una variante dell'architettura a tre livelli. La Three-Tier Architecture viene utilizzata principalmente per garantire modularità, scalabilità e manutenibilità nelle applicazioni. In particolare questa architettura garantisce :

- **Separazione delle Responsabilità:** Ogni livello ha un ruolo ben definito e svolge un compito specifico, questa separazione rende il codice più leggibile e manutenibile
- **Maggiore manutenibilità:** Se è necessario modificare un livello (es. cambiare il database da MySQL a PostgreSQL), si può farlo senza impattare gli altri livelli.
- **Facilità di Testing:** Possiamo testare il Service Layer senza bisogno di un database reale, grazie all'uso di mock (come abbiamo fatto nella parte di testing)

Questa architettura si integra bene con sistemi REST perché separa chiaramente la logica di business dall'interfaccia utente e dal livello di persistenza.

10.3 Struttura Schematizzata Back-End



Con questo schema viene riassunto il modo di comunicare fra i vari componenti del Back-End, rendendo l'visione del suo funzionamento più semplice e chiara.

10.4 Tecnologie Adottate

In questa sezione, descriviamo le tecnologie, i framework e gli strumenti utilizzati per lo sviluppo del software.

10.5 JavaFX

JavaFX è un framework di sviluppo per la creazione di interfacce grafiche in Java.



Logo di JavaFx

Volevamo sviluppare un'applicazione desktop performante ed efficiente, e grazie alle tecnologie avanzate rispetto a quelle più datate come Swing o AWT, **JavaFX è diventata la nostra scelta principale per un'applicazione desktop in grado di soddisfare i requisiti richiesti del cliente.**

JavaFx è stato progettato per sostituire Swing come toolkit standard per l'interfaccia grafica di Java, offrendo un approccio più moderno allo sviluppo dell'interfaccia utente, più in particolare:

- **Comunità ed Ecosistema:** JavaFX vanta una comunità dedicata e una vasta gamma di librerie e strumenti che ne migliorano la funzionalità. (Come **CalendarFx** che abbiamo usato)
- **Supporto e Sviluppo:** JavaFX era inizialmente parte del JDK, ma è stato separato a partire dal JDK 11. Da allora, è stato mantenuto come un progetto separato sotto l'egida di OpenJFX, il che ha permesso uno sviluppo continuo e contributi da parte della comunità.
- **Funzionalità Avanzate per l'Interfaccia Utente:** JavaFX offre un insieme **completo di controlli per l'interfaccia utente, un supporto al CSS (che in Swing non era presente) e supporto per animazioni e grafica**, rendendolo adatto alla creazione di applicazioni visivamente accattivanti.
- **FXML** JavaFX supporta gli FXML, un linguaggio basato su XML che permette agli sviluppatori di definire interfacce utente non scrivendo codice nel sorgente e di conseguenza rendere il codice più pulito e leggibile.

Tuttavia, anche se la sua popolarità è stata in parte oscurata da altri framework come i framework JavaScript (ad esempio, React, Angular) per le applicazioni web e strumenti per lo sviluppo mobile come Flutter e React Native. **JavaFX rimane comunque una scelta valida per lo sviluppo di applicazioni desktop in Java, grazie alla sua integrazione con il linguaggio e alle potenti**

capacità grafiche, come il supporto per animazioni, effetti visivi e la gestione avanzata dei controlli dell'interfaccia utente.

10.6 Jakarta Enterprise Edition

Jakarta Enterprise Edition è un'evoluzione di JavaEE, è un insieme di specifiche che estende Java SE con specifiche per funzionalità aziendali come il calcolo distribuito e i servizi web. Le applicazioni Jakarta EE vengono eseguite su runtime di riferimento, che possono essere microservizi o server di applicazioni, i quali gestiscono transazioni, sicurezza, scalabilità, concorrenza e la gestione dei componenti che stanno distribuendo. Jakarta EE include diverse specifiche che servono a scopi differenti, come la generazione di pagine web, la lettura e scrittura da un database in modo transazionale, e la gestione di code distribuite.

Le API di Jakarta EE comprendono diverse tecnologie che estendono le funzionalità delle API di base di Java SE, come Jakarta Enterprise Beans, connettori, servlet, Jakarta Server Pages e diverse tecnologie per i servizi web come **Jax-RS. Jakarta RESTful Web Services, e le sue implementazioni**. Jakarta RESTful Web Services è una specifica dell'API Jakarta EE che fornisce supporto nella creazione di servizi web secondo il pattern architetturale Representational State Transfer (REST). JAX-RS utilizza annotazioni, introdotte in Java SE 5, per semplificare lo sviluppo e il deployment di client e endpoint dei servizi web.



Logo di Jakarta EE

Perché Jakarta RESTful Web Services?

Jakarta RESTful Web Services è ampiamente utilizzato per creare API REST in applicazioni Java, grazie alla sua semplicità, flessibilità e integrazione con l'ecosistema Java

- **Standardizzazione e Portabilità:** JAX-RS è un'API, quindi possiamo scegliere tra diverse implementazioni (ad esempio, **Jersey**, **RESTEasy**, **Apache CXF**) senza dover riscrivere il codice.
- **Semplicità e leggibilità:** JAX-RS utilizza annotazioni come **@Path**, **@GET**, **@POST**, **@PUT**, **@DELETE**, **@Consumes**, e **@Produces** per definire risorse e metodi HTTP in modo chiaro e conciso. Questo riduce la quantità di codice boilerplate e rende il codice più leggibile.
- **Standardizzazione:** Essendo una specifica standard definita dalla piattaforma Jakarta EE, JAX-RS garantisce interoperabilità e portabilità tra diverse implementazioni e server applicativi.

Le più note implementazioni di JAX-RS sono frameworks come

- **Eclipse Jersey** è una delle principali implementazioni di JAX-RS ed è stato scelto proprio questa implementazione nel nostro software, per la sua alta conformità ad JAX-RS e offre tutte le funzionalità standard per la creazione di servizi RESTful.

10.7 SceneBuilder

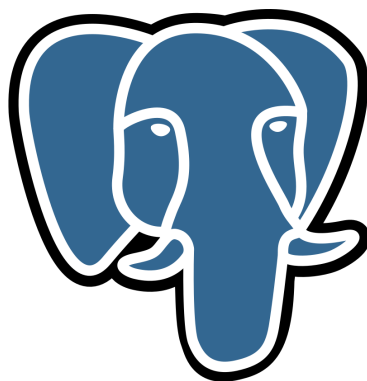
Scene Builder è uno strumento di **progettazione visiva per JavaFX** che consente agli sviluppatori di creare interfacce utente in modo rapido e intuitivo, evitando la necessità di scrivere manualmente il codice XML per i file FXML. In combinazione con JavaFX, questo software, appositamente sviluppato per tale scopo, permette di progettare interfacce grafiche in modo più efficiente e performante, senza dover redigere manualmente i file FXML.



Logo di SceneBuilder

10.8 PostgreSQL

PostgreSQL è una potente **base di dati relazionale gratuita ed open source che usa il linguaggio SQL** standard ma non disdegnando delle aggiunte utili per conservare e scalare i carichi di dati più complessi (è gergalmente definito un "dialetto"). PostgreSQL ha una forte reputazione di RDBMS robusto, affidabile, estensibile. Esso 'e eseguibile su tutti i principali sistemi operativi e garantisce le proprietà ACID (atomicità, consistenza, isolamento, durabilità) per dati e transazioni. Inoltre fornisce anche utili add-on per estendere le capacità della base di dati, è disponibile sia attraverso riga di comando che GUI



Logo di PostgreSQL

Perchè PostgreSQL? PostgreSQL fornisce numerose funzionalità utili per lo sviluppo delle applicazioni, la protezione dell'integrità dei dati e la costruzione di ambienti tolleranti al guasto. Permette anche di scrivere procedure e funzioni procedurali in differenti linguaggi e per differenti linguaggi di programmazione (PL/pgSQL, PL/Tcl, PL/Perl, PL/Python). Tra le funzionalità notevoli, si possono annoverare:

- Supporto ad una vasta quantità di tipi di dato
- Elevata sicurezza
- Supporto sempre gratuito
- Il linguaggio procedurale PL/pgSQL 'e pienamente compatibile con il linguaggio procedurale di altre basi di dati, facilitando eventuali future migrazioni
- Ampia estensibilità
- Affidabilità e ripresa dal disastro

10.9 Docker

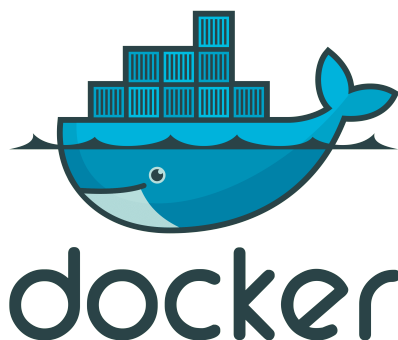
Docker è un insieme di servizi Platform as a Service (PaaS) che utilizza la virtualizzazione a livello di sistema operativo del kernel per eseguire programmi in ambienti isolati e distribuibili chiamati container, i quali sono gestiti dal Docker Engine.

I container sono leggeri e al loro interno conterranno tutto il necessario per l'esecuzione del software, senza preoccuparsi di quale macchina sta eseguendo il Docker Engine.

Questi container sono configurabili attraverso l'uso di file di configurazione appositi.

Compose, inizialmente un add-on per la piattaforma Docker, è stato successivamente incluso al suo interno, e consente attraverso un singolo file YAML la gestione e la comunicazione reciproca di più container.

I container sono un'istanza di un'immagine Docker, ossia un template di sola lettura che istruisce su come il container debba essere creato.



Logo di Docker

Perché Docker?

- **Isolamento e Consistenza dell'Ambiente:** Docker ha permesso di isolare il nostro software in un container, che include tutte le dipendenze necessarie per eseguirlo su qualsiasi macchina, come librerie, configurazioni di sistema, e il runtime. Questo significa che il mio software può essere eseguito su qualsiasi macchina, senza dover affrontare problemi di compatibilità tra ambienti diversi.
- **Facilità di Configurazione:** Grazie Docker, abbiamo descritto l'ambiente della nostra applicazione in modo semplice utilizzando un file chiamato Dockerfile. Questo file definisce tutte le istruzioni per costruire il container, come l'installazione delle dipendenze e la configurazione dell'applicazione.

- **Portabilità:** Docker migliora la portabilità del software, consentendo di eseguire l'applicazione su qualsiasi server che supporti Docker, sia esso un server locale, una macchina virtuale o un server nel cloud. Infatti noi dopo aver installato una macchina virtuale su Amazon, abbiamo trasferito il nostro progetto tramite terminale e lo abbiamo avviato utilizzando docker-compose opportunamente configurato. Questo ci ha permesso di esporre il nostro server su un indirizzo pubblico, rendendolo accessibile dall'esterno.

10.10 JavaScript

JavaFX fornisce il **componente WebView**, che permette di visualizzare e interagire con contenuti web all'interno di un'applicazione Java. **Grazie al motore WebEngine, è possibile eseguire codice JavaScript e consentire la comunicazione bidirezionale tra Java e JavaScript.**

Questa integrazione permette di combinare la potenza di Java con la flessibilità di JavaScript e delle tecnologie web, offrendo un'esperienza utente moderna e interattiva.



Figure 1: Logo di JavaScript

10.10.1 Perché JavaScript?

Abbiamo integrato JavaScript nel nostro progetto JavaFX per rispondere alle specifiche esigenze del committente. In particolare, l'utilizzo di JavaScript si è reso necessario per garantire un'efficace integrazione con le mappe, funzionalità richiesta dal committente.

L'integrazione tra Java e JavaScript è stata impiegata in due contesti principali:

- **1.Creazione dell'annuncio :** l'utente può interagire con la mappa per selezionare la posizione dell'immobile, **migliorando l'esperienza d'uso e la precisione nell'inserimento dei dati geografici.**
- **2.Ricerca degli annunci :** la mappa consente di visualizzare gli immobili disponibili, **offrendo un'interfaccia interattiva e intuitiva per la navigazione e la selezione delle proprietà.**

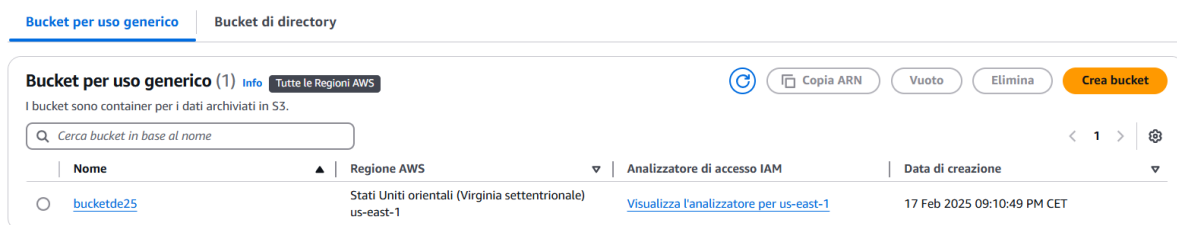
L'adozione di JavaScript ha permesso di combinare la solidità e la struttura di Java con la flessibilità delle tecnologie web, garantendo un'esperienza utente avanzata e conforme ai requisiti del progetto

In particolare:

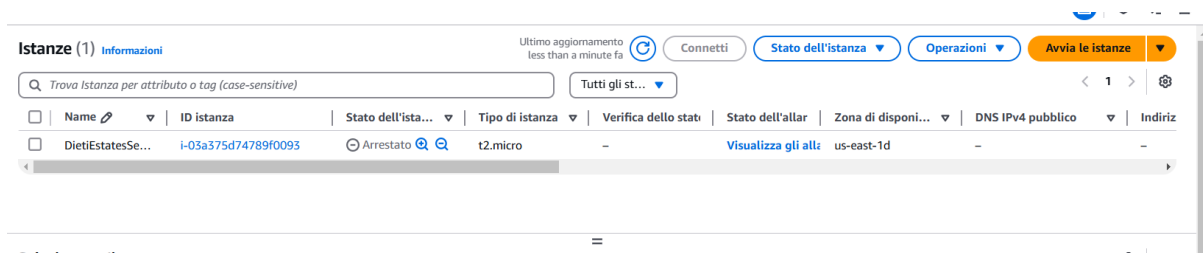
- Abbiamo utilizzato la mappa di **OpenStreetMap**, un progetto collaborativo che fornisce mappe geografiche libere e open-source.
- Per il caricamento e la visualizzazione della mappa, ci siamo avvalsi di **Leaflet**, una libreria **JavaScript** open-source progettata per la gestione di mappe interattive.
- Per ottenere le coordinate geografiche necessarie all'inserimento del marker sulla mappa, abbiamo utilizzato le API del servizio **HERE**, che forniscono latitudine e longitudine a partire da un indirizzo. I dati restituiti da **HERE** sono stati elaborati da JavaScript, che ha provveduto a posizionare il marker sulla mappa.

10.11 Altri servizi cloud usati

- **Amazon EC2:** Amazon Elastic Compute Cloud (EC2) è un servizio di cloud computing fornito da Amazon Web Services (AWS) che consente di lanciare e gestire istanze di macchine virtuali. Abbiamo scelto questo servizio perché possiamo facilmente scalare le risorse computazionali in base alle esigenze dell'applicazione. Inoltre, EC2 offre una vasta gamma di tipi di istanze con diverse configurazioni hardware, permettendoci di scegliere quella più adatta alle nostre esigenze di prestazioni e costo.
- **Amazon S3:** Amazon Simple Storage Service (S3) è un servizio di storage di oggetti che consente di archiviare e recuperare dati di qualsiasi dimensione in modo sicuro ed efficiente. Abbiamo utilizzato Amazon S3 per memorizzare immagini e loghi dell'applicazione, poiché offre una durabilità elevata, scalabilità e facilità di integrazione con altri servizi AWS.



Bucket per le immagini



Istanza EC2

11 Motivazione delle scelte di design

11.1 Intro



Intro del sistema

Abbiamo deciso di introdurre quando parte l'applicazione un piccolo caricamento da far mostrare all'utente al fine di migliorarne l'usabilità complessiva del prodotto finale e di migliorare il suo coinvolgimento durante l'attesa evitando frustrazioni, rendendo l'attesa più piacevole e dare un'identità visiva più curata.

11.2 Colori del software

Per il nostro prodotto software abbiamo scelto principalmente due combinazioni di colori: il **blu** e il **bianco**, questi colori trasmettono sensazioni ben precise. Il blu perchè dal punto di vista psicologico, **ha un effetto rilassante e antistress e si ritiene che stimoli l'inconscio e l'intuizione**. Il bianco, invece, **rappresenta semplicità e pace**. Questa combinazione di colori crea un ambiente visivo che ispira tranquillità, chiarezza e fiducia, elementi fondamentali per un'esperienza utente piacevole ed efficace.

Color Contrast Checker

Calculate the contrast ratio of text and background colors.

The screenshot shows the Color Contrast Checker interface. On the left, there are input fields for 'Text color' (set to #FFFFFF) and 'Background color' (set to #011638). Below these, the 'Contrast' section displays a large green box with the value '17.90' and the word 'Super' with five stars. A table below shows 'Small text' and 'Large text' both with three stars. A note at the bottom states 'Great contrast for all text sizes.' On the right, a dark blue rectangular preview shows the text 'Quote n. 21' in white, followed by a quote and the name 'Norm Crosby'.

Text color	Background color
#FFFFFF	#011638

Contrast

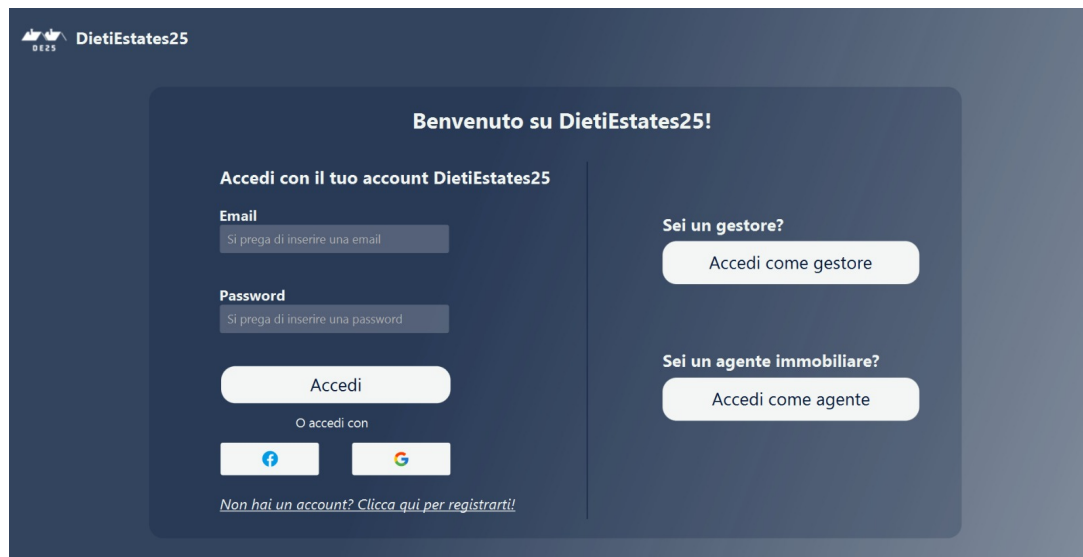
17.90 Super ★★★★★

Text size	Contrast
Small text	★★★
Large text	★★★

Great contrast for all text sizes.

Contrast ratio dei colori principali del sistema

11.3 Pagina di benvenuto



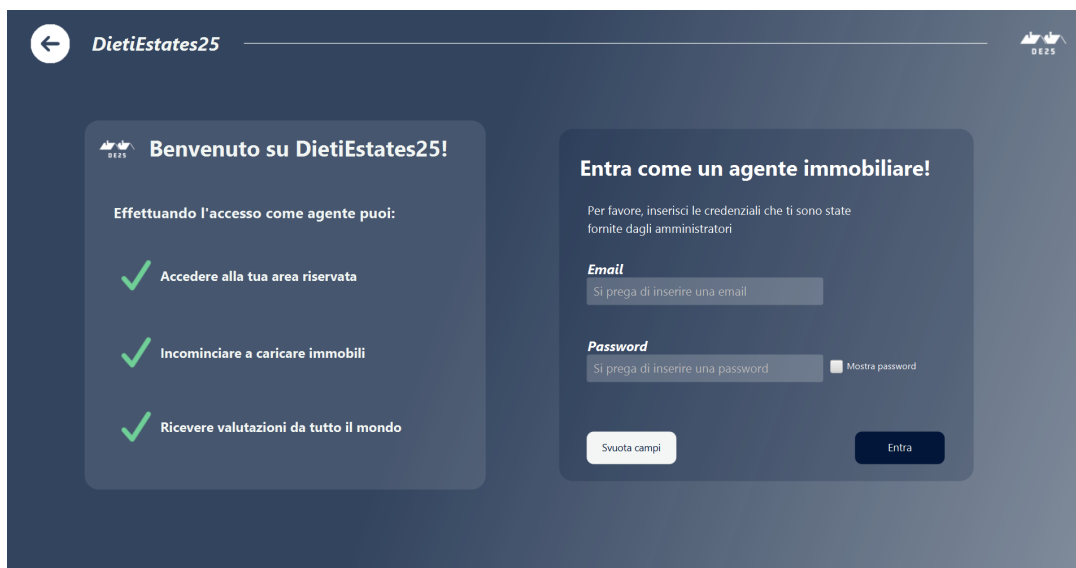
Qui è rappresentata la Home del sistema. All'avvio del software, questa è la prima schermata che viene mostrata all'utente.

Abbiamo scelto di introdurre una pagina di **benvenuto invece della classica pagina di ricerca**, seguendo le convenzioni delle applicazioni moderne, che **spesso presentano un launcher iniziale per orientare l'utente e permettergli di scegliere l'azione da intraprendere**. Un'altra motivazione alla base di questa scelta è che non tutti gli utenti del sistema hanno come obiettivo principale la ricerca di un immobile. Ad esempio, gestori e agenti immobiliari potrebbero avere esigenze diverse. Per questo motivo, la Home funge da punto di riferimento chiaro per tutti, **guidando ogni tipologia di utente verso la sezione più adatta alle proprie necessità**.

11.4 Analisi del design

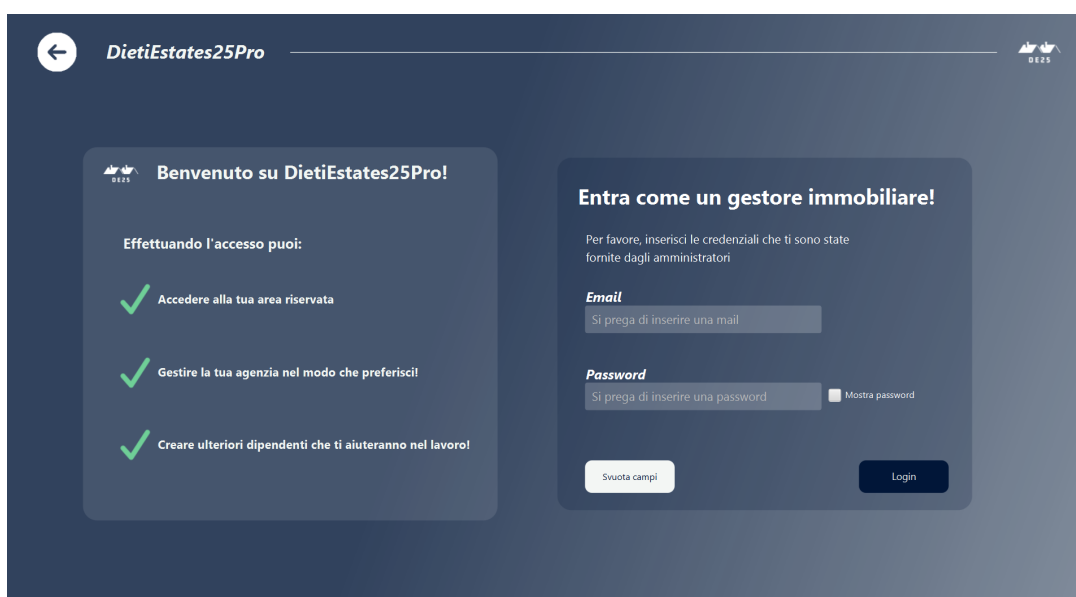
Abbiamo deciso di mantenere una gerarchia visiva semplice nella pagina di benvenuto di DietiEstates25, facendo in modo che i clienti comprendano subito le opzioni di login standard e registrazione. **Il pulsante 'Accedi' è posto in evidenza come l'azione principale**, facilmente visibile appena si apre la pagina. Gli altri pulsanti, di colore bianco, rappresentano azioni secondarie destinate agli utenti interessati alla parte professionale del sistema. **Questi bottoni sono meno evidenti per non distogliere l'attenzione dalla principale azione di login**.

11.5 Pagine di accesso



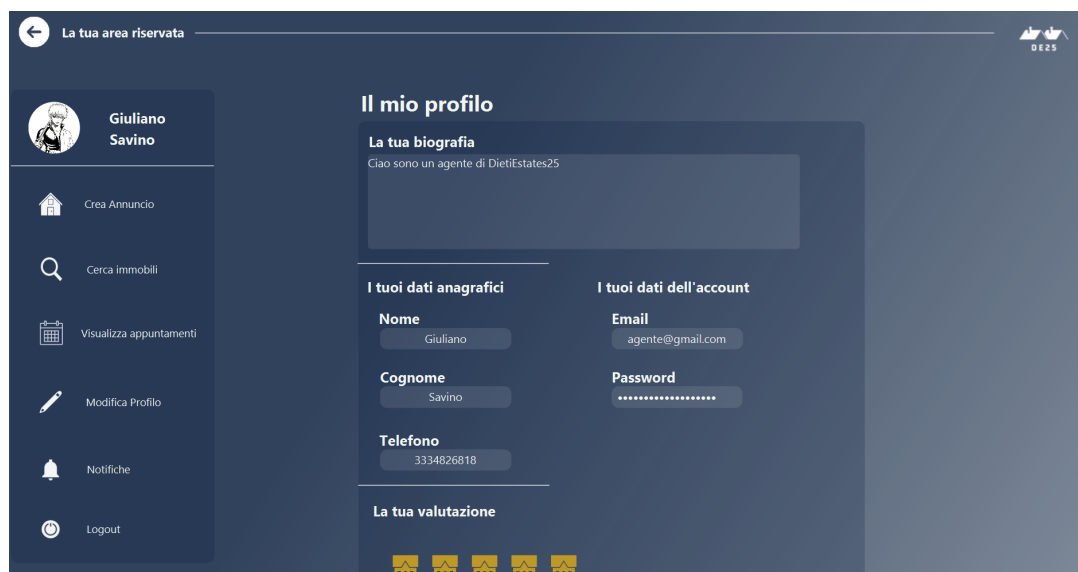
Per la pagine di accesso degli agenti e dei gestori abbiamo utilizzato un design che promuove il **Principio della regione comune**, al fine di migliorare l'usabilità del prodotto. Grazie a questo principio si delimitano due principali regioni:

- **Regione dei vantaggi** : In questa regione delimitiamo i vantaggi che l'utente ha ad accedere e cosa può fare quando accede al sistema.
- **Regione d'accesso**: Regione dedicata completamente all'accesso dell'agente che spiega che per accedere deve aver ricevuto delle credenziali dagli amministratori.



11.6 Aree riservate

Per le aree riservate agli utenti, abbiamo adottato un design che migliora l'usabilità del prodotto finale, basandoci sul **Principio della regione comune** e sul **Principio di prossimità**.



Grazie a questi principi, abbiamo suddiviso la pagina in tre aree principali:

- **Area dati:** È la sezione principale della pagina, in cui vengono mostrati i dati dell'utente, permettendogli di averli sempre sotto controllo. Le informazioni sono organizzate in blocchi distinti e, grazie al **Principio di prossimità**, l'utente può comprendere immediatamente a cosa si riferiscono.
- **Area azioni:** Qui sono raccolte tutte le funzionalità disponibili per l'agente immobiliare. Le operazioni più rilevanti e frequenti sono posizionate in modo da essere facilmente accessibili, riducendo il numero di movimenti necessari con il mouse.
- **Area annunci recenti:** In questa sezione vengono elencati gli annunci più recenti pubblicati dall'agente. Ogni annuncio è chiaramente separato dagli altri per regione al fine di migliorare la leggibilità e l'usabilità del prodotto.

11.7 Crea annuncio

Per la pagina di creazione annuncio, **abbiamo deciso di adottare un design che minimizzi gli errori, consentendo all'agente di caricare i suoi immobili in modo rapido e piacevole. Abbiamo suddiviso la pagina in diverse aree invece di creare un'unica lunga pagina scrollabile**, al fine di migliorare l'esperienza dell'utente. In questo modo, l'agente può cambiare facilmente contesto e sapere immediatamente dove dirigersi, senza dover navigare avanti e indietro nella pagina.

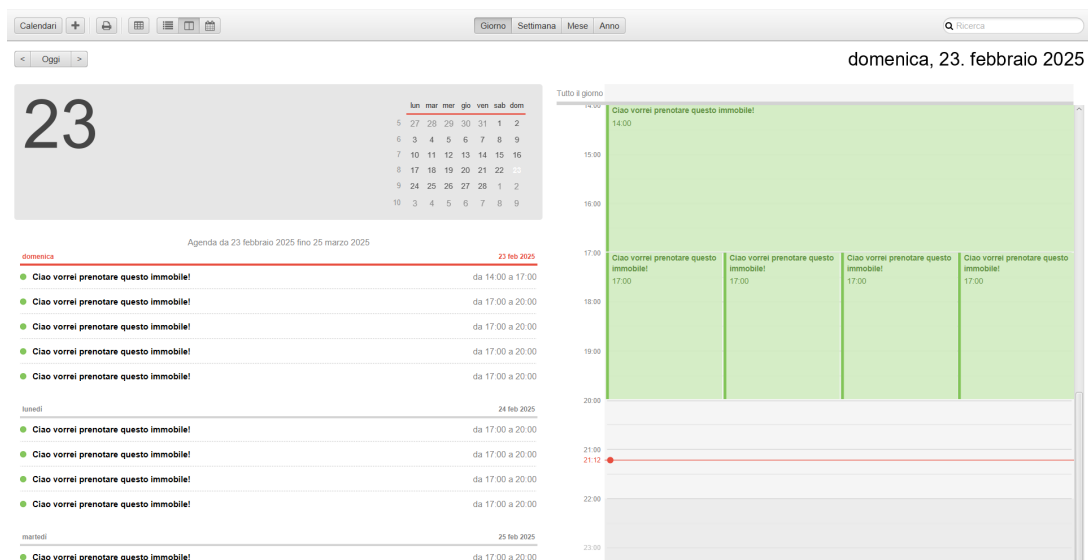
Abbiamo adottato questo design sfruttando il principio della regione comune dove in ogni regione vi sono opzioni correlate fra di loro. Abbiamo anche scelto di minimizzare al minimo di errori facendo scegliere l'input all'utente tramite dei radio-button.

Inoltre come possiamo notare sopra in tutte le pagine della creazione dell'annuncio il bottone avanti è bloccato, questo design ha diversi vantaggi:

- **Chiarezza per l'utente:** Gli utenti sanno esattamente cosa devono fare per procedere. Questo evita confusioni e riduce il rischio di errori, in quanto non possono andare oltre senza aver completato tutti i passaggi necessari.
- **Gestione delle aspettative:** Gli utenti sono incentivati a procedere passo per passo, il che crea una percezione di ordine e controllo del processo, riducendo la frustrazione.
- **Riduce frustrazione:** Riteniamo che una migliore esperienza utente consista nel permettere al cliente di correggere gli errori man mano che li incontra, piuttosto che ritrovarsi alla fine con errori sparsi in tutta la pagina dopo aver completato il modulo di creazione dell'annuncio. In questo modo, l'utente può risolvere subito ogni problema, evitando frustrazioni e migliorando la fluidità dell'interazione.

11.8 Visualizza appuntamenti

Il design della pagina è stato sviluppato tramite la libreria di JavaFx chiamata **JavaFX Calendar**.



L'interfaccia consente di navigare tra gli appuntamenti con facilità, grazie a pulsanti di navigazione per cambiare visualizzazione (giorno, settimana, mese, anno). Inoltre, gli eventi vengono rappresentati con colori distintivi in base alla loro tipologia, facilitando la comprensione a colpo d'occhio. L'utente può interagire con gli appuntamenti tramite clic per visualizzare maggiori dettagli o apportare modifiche. L'integrazione con il database assicura che ogni aggiornamento venga sincronizzato in tempo reale

11.9 Modifica profilo

Il design della pagina di modifica di profilo è stato sviluppato seguendo il **Principio della regione comune** andando a dividere la pagina in due principali sezioni:

Modifica profilo

0825



Sasha Izzo

La tua biografia

Ciao sono un agente di DietiEstates.

Aggiorna

Modifica i tuoi dati!

1. I tuoi dati anagrafici

Nome

Sasha

Cognome

Izzo

Prefisso

+39

Telefono

3664398508

2. I tuoi dati dell'account DietiEstates25

Email

sasha@gmail.com

Password

Modifica password

Aggiorna

- **Area visualizzazione:** Questa sezione, situata nella parte sinistra della pagina, è dedicata alla presentazione dell'agente immobiliare. **Abbiamo scelto di collocare insieme la sua foto e la biografia perché entrambi gli elementi svolgono un ruolo fondamentale nella comunicazione visiva e nella costruzione della sua identità professionale.** La foto permette agli

utenti di associare un volto all'agente, mentre la biografia fornisce informazioni essenziali sul suo percorso e le sue competenze. Unendo questi due aspetti in un'unica area, garantiamo un'esperienza utente più chiara e immediata, agevolando la percezione e la comprensione del profilo dell'agente.

- **Area Dati:** Questa sezione raccoglie tutte le informazioni personali dell'agente, sia di natura anagrafica che relative al suo account. Abbiamo scelto di organizzare i dati in una disposizione verticale per garantire un layout chiaro, ordinato e facilmente leggibile. Questa struttura facilita la consultazione delle informazioni, migliorando l'esperienza utente e rendendo più intuitiva la gestione dei dati.

Abbiamo scelto di rendere i bottoni **non cliccabili fino a quando l'utente non effettua una modifica ai propri dati per garantire un'esperienza utente più chiara ed efficiente**. Questo approccio presenta diversi vantaggi

- **Migliora la chiarezza dell'interfaccia:** evita che l'utente prema accidentalmente un pulsante senza aver apportato modifiche, riducendo possibili confusioni.
- **Previene azioni inutili:** impedisce l'invio di dati identici a quelli già registrati, evitando richieste superflue al sistema.
- **Guida l'utente nel processo di modifica:** evidenziando l'attivazione dei bottoni solo dopo una modifica, l'interfaccia comunica in modo intuitivo quando un'azione è necessaria.

11.10 Creazione account

Per il design della pagina di creazione degli account abbiamo sfruttato il **Principio della regione comune** dividendo la pagina in due principali regioni:

Crea gestori

Crea i tuoi gestori su DietiEstates25Pro!

Una volta creati i tuoi gestori possono:

- ✓ Accedere alla loro area riservata
- ✓ Creare agenti immobiliari
- ✓ Aiutarti a coordinare le azioni della tua azienda

Crea i tuoi gestori!

1. Inserisci i dati anagrafici del gestore

Nome: es. Giulio

Cognome: es. Rossi

Codice fiscale: es. GURSS00B27F839E

Prefisso: +39 Telefono: es. 3512467788

2. Inserisci i dati per il suo account

Email: es. giuliorossi00@gmail.com

Password (Almeno 8 caratteri): Es. La mia password ☐ Mostra password

Svuota campi Crea

- **Regione dei vantaggi:** In questa regione sono indicati tutti i vantaggi e le conseguenze che l'amministratore avrà quando creerà l'account per i gestori.

- **Regione dei dati:** In questa regione l'amministratore immetterà i dati per la creazione dell'account. Abbiamo scelto di organizzare i dati in una disposizione verticale per garantire un layout chiaro, ordinato e facilmente consultabile. Questa struttura facilita la consultazione delle informazioni, migliorando l'esperienza utente e rendendo più intuitiva la gestione dei dati.

In questo design abbiamo deciso di mantenere il pulsante "Crea" **sempre disponibile e visibile**, in questo modo l'amministratore ha un maggiore controllo sull'operazione e può decidere liberamente quando procedere. Questo gli permette di rivedere i dati inseriti in qualsiasi momento, correggere eventuali errori senza restrizioni e avere un riscontro immediato sulla compilazione dei campi.

Inoltre, la visibilità costante del pulsante riduce il rischio di frustrazione, evitando che l'utente debba completare tutti i campi prima di rendersi conto di eventuali problemi. Questo approccio favorisce un'interazione più fluida, intuitiva e consapevole, migliorando l'esperienza utente e riducendo potenziali errori.

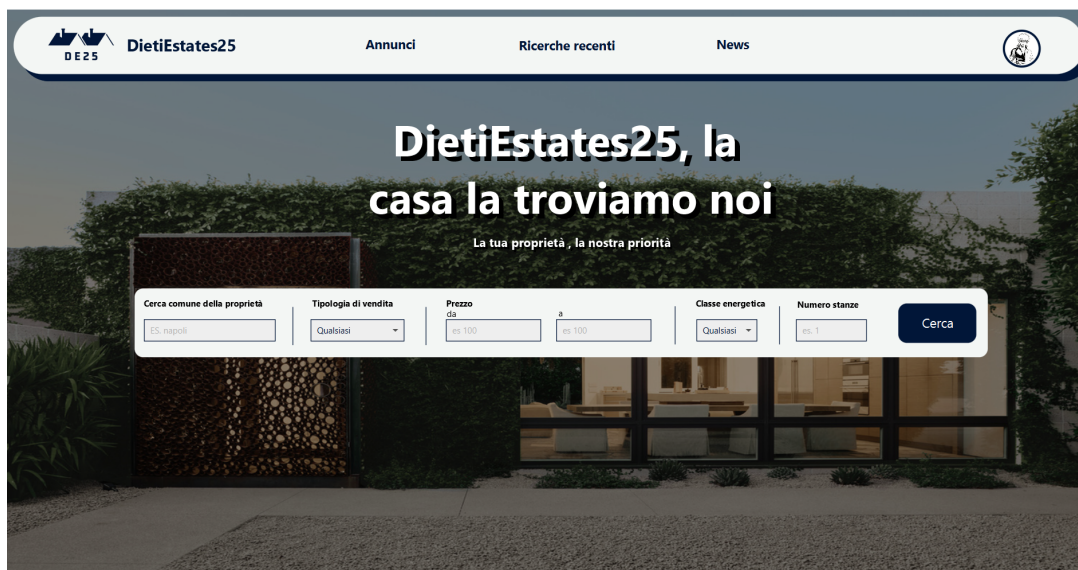
11.11 Visualizza notifiche

Per il design della visualizzazione delle notifiche, abbiamo adottato il **Principio della regione comune**, suddividendo ogni notifica in aree distinte, ciascuna con i propri dati e un colore specifico. Questa scelta permette all'utente di riconoscere immediatamente la tipologia di notifica, migliorando la chiarezza e la fruibilità dell'interfaccia.



11.12 Home di DietiEstates25

Per la home di DietiEstates25, abbiamo adottato un design che minimizza la complessità della pagina, rendendola immediatamente chiara e intuitiva. Una scritta in primo piano **comunica in modo diretto lo scopo della schermata, evitando ambiguità**. La barra di ricerca è ben visibile fin da subito, **senza che l'utente debba navigare attraverso menù complessi**, facilitando così l'inizio della ricerca di un immobile. Inoltre, la gerarchia visiva è studiata per guidare l'utente, evidenziando il **pulsante "Cerca" come azione principale**, mentre le opzioni secondarie sono presentate con un design meno prominente ma comunque accessibile.



11.13 Ricerche passate

Per il design delle ricerche passate abbiamo deciso di strutturare ogni ricerca in card diverse questo comporta diversi vantaggi:

- **Maggiore leggibilità:** Le informazioni vengono organizzate in blocchi chiari e distinti, evitando il sovraccarico visivo.
- **Navigazione intuitiva:** Ogni card rappresenta un'entità autonoma, facilitando la scansione visiva e la selezione.

- **Facilità di confronto:** Gli utenti possono confrontare le opzioni rapidamente senza perdere il contesto.

Ricerche passate



Case-Appartamenti
Comune: Qualsiasi
Stato: VENDITA
Prezzo Minimo: Qualsiasi
Prezzo Massimo: Qualsiasi
Numero stanze: Qualsiasi
Classe Energetica: Qualsiasi
[Avvia ricerca](#)

Case-Appartamenti
Comune: Napoli
Stato: Qualsiasi
Prezzo Minimo: Qualsiasi
Prezzo Massimo: Qualsiasi
Numero stanze: Qualsiasi
Classe Energetica: Qualsiasi
[Avvia ricerca](#)

Case-Appartamenti
Comune: Qualsiasi
Stato: VENDITA
Prezzo Minimo: Qualsiasi
Prezzo Massimo: Qualsiasi
Numero stanze: Qualsiasi
Classe Energetica: Qualsiasi
[Avvia ricerca](#)

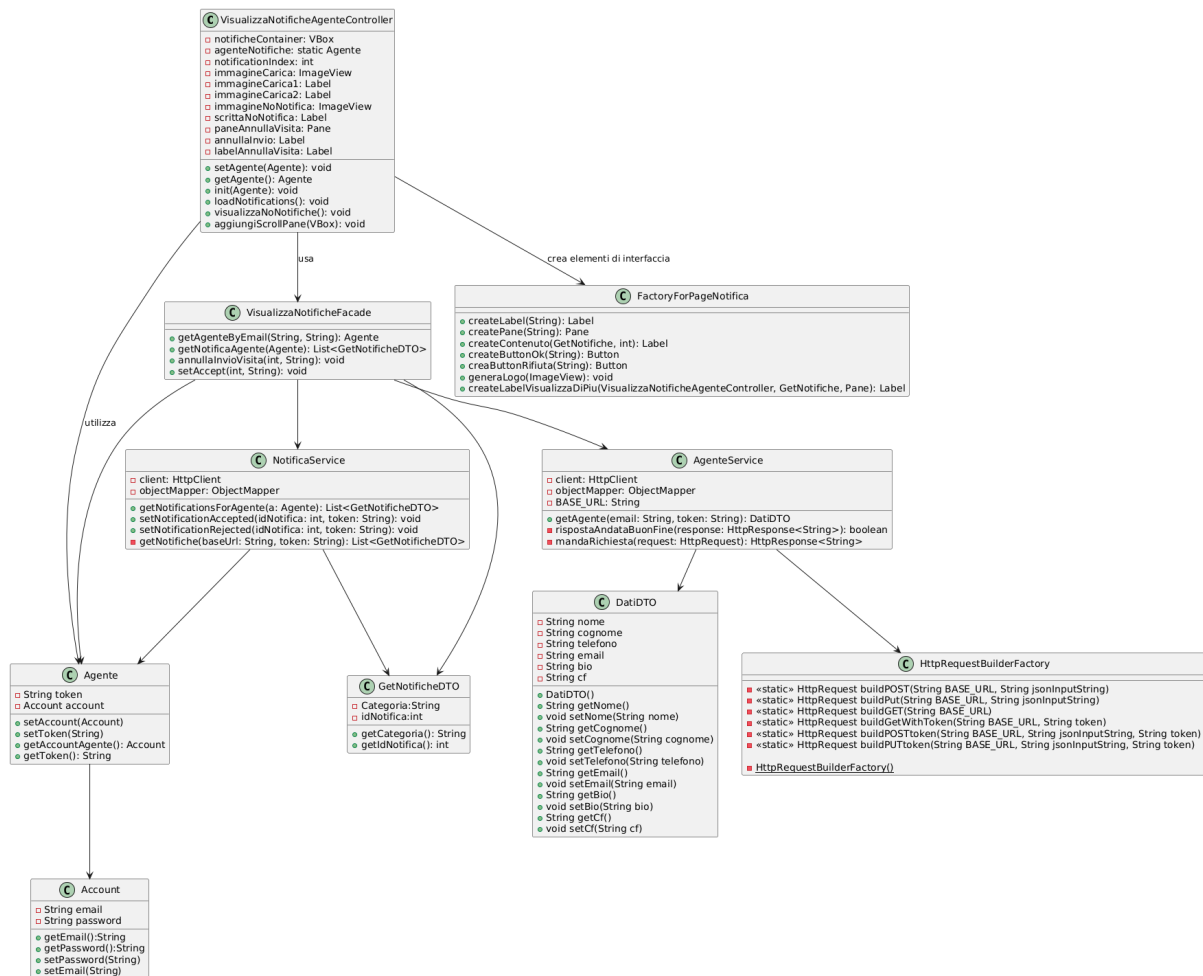


12 Diagrammi delle classi di design

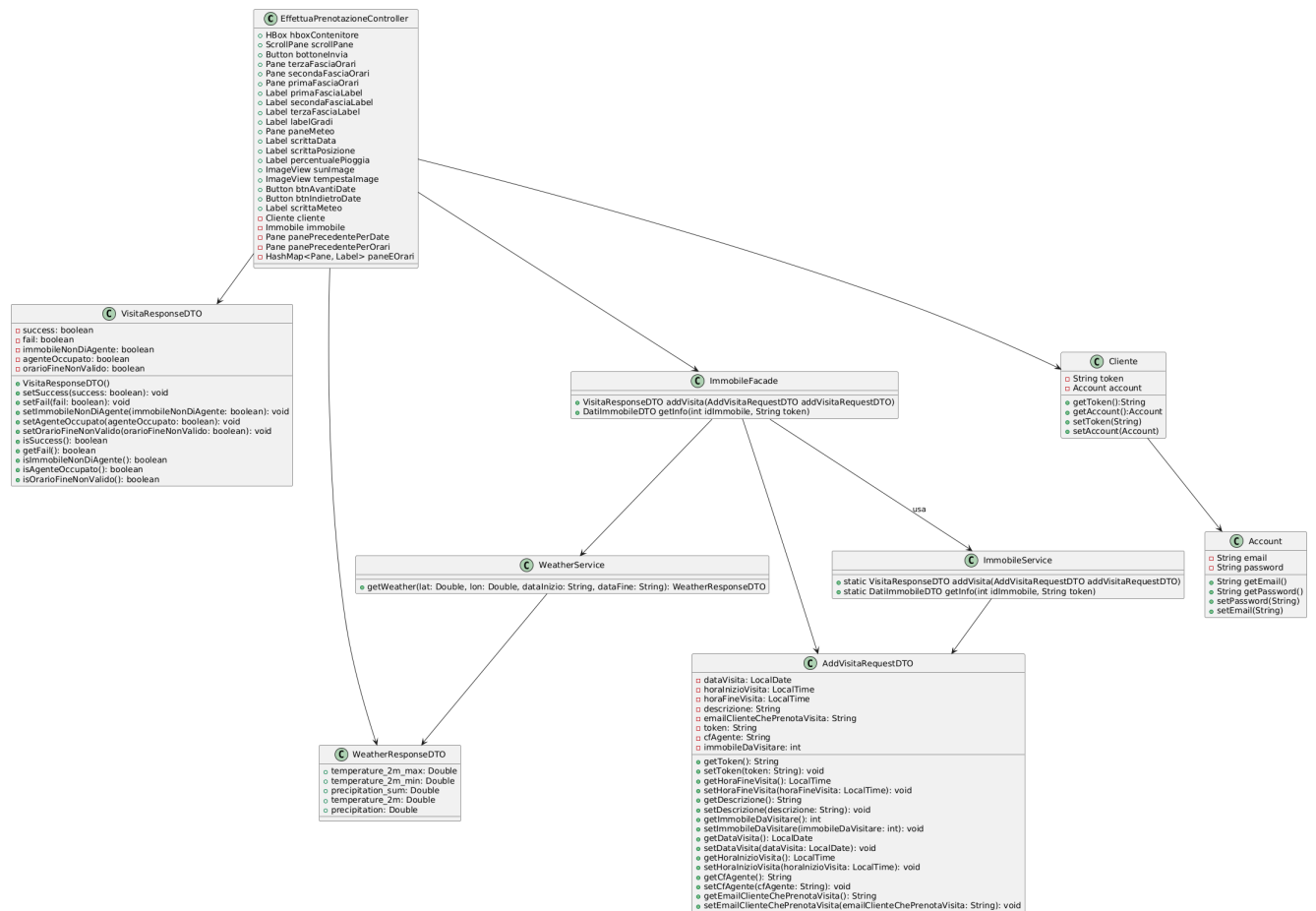
Di seguito sono elencati i diagrammi delle classi che rappresentano le interazioni tra le principali classi del sistema.

12.1 Gestione delle notifiche

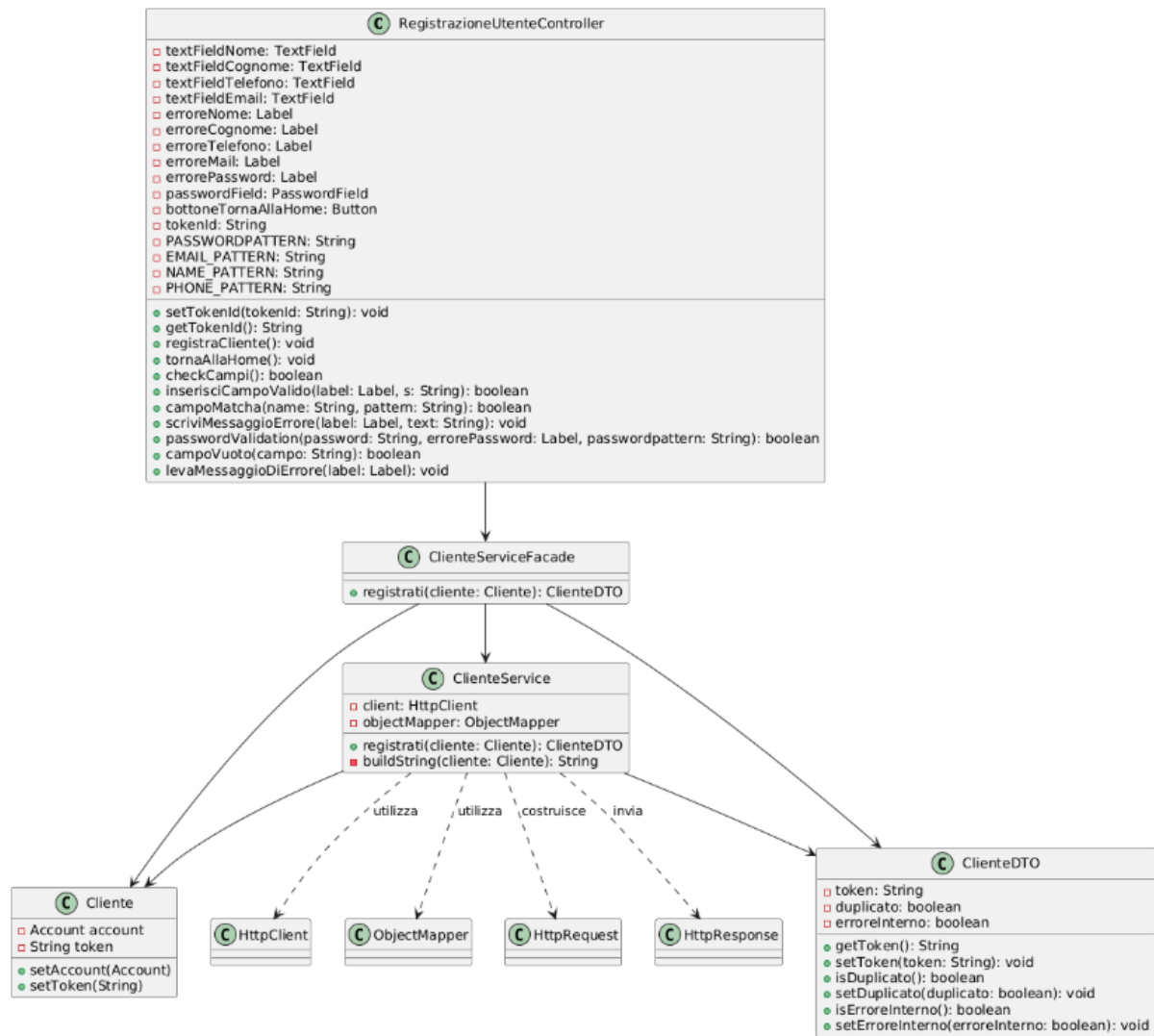
Le immagini poste di seguito verranno mostrate divise al fine di migliorare la leggibilità.



12.2 Effettua prenotazione



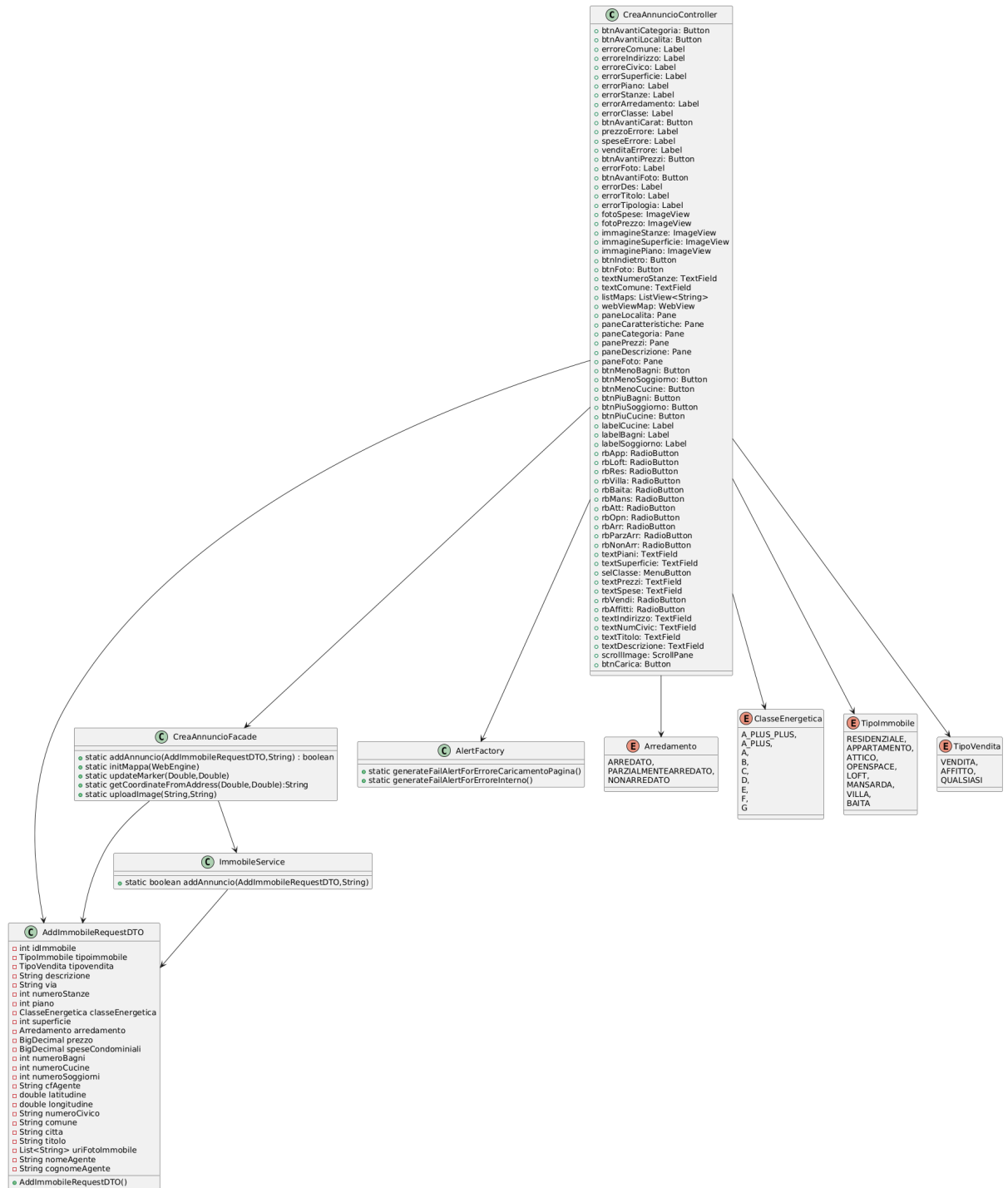
12.3 Registrazione



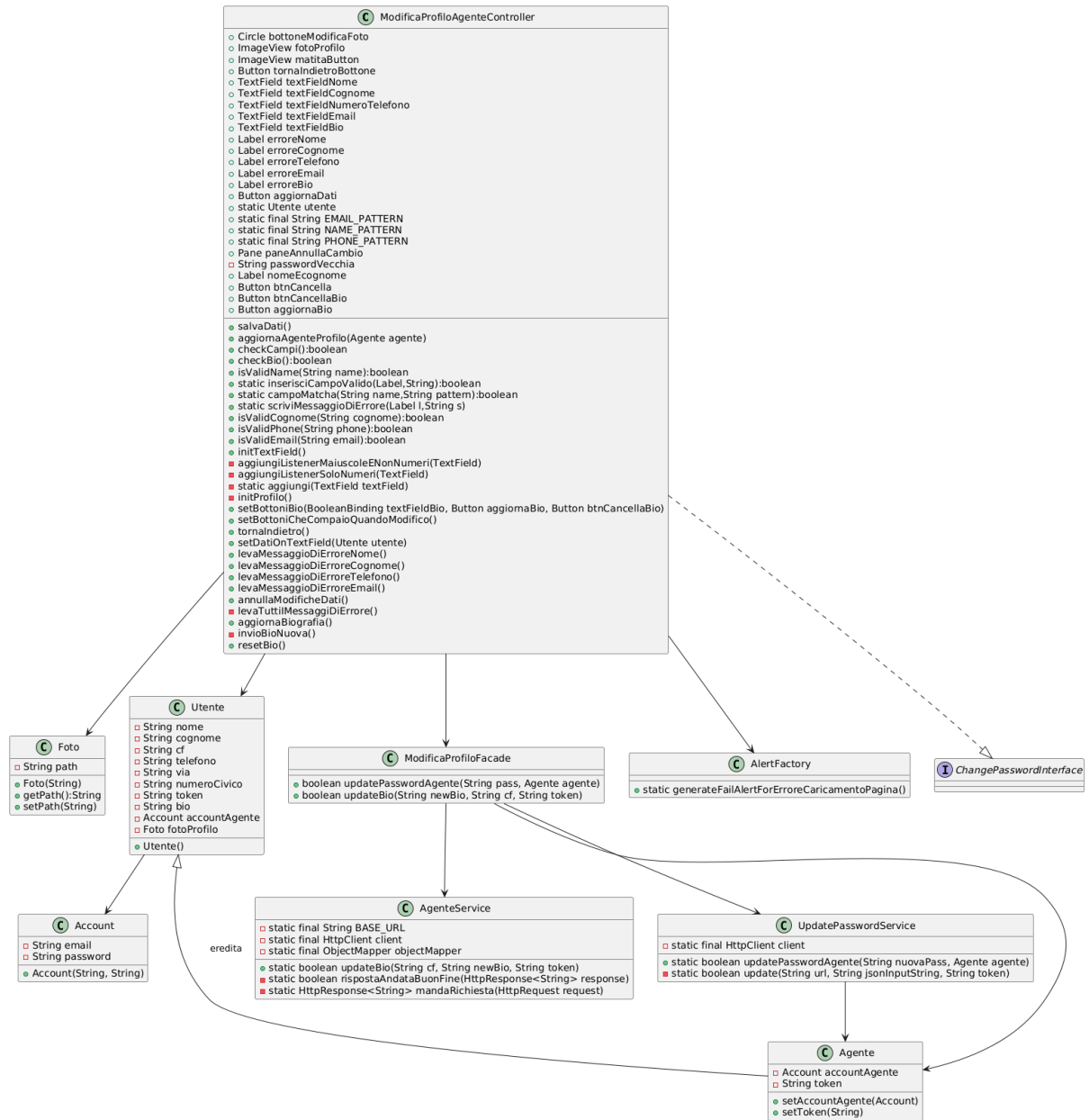
12.4 Cambio password



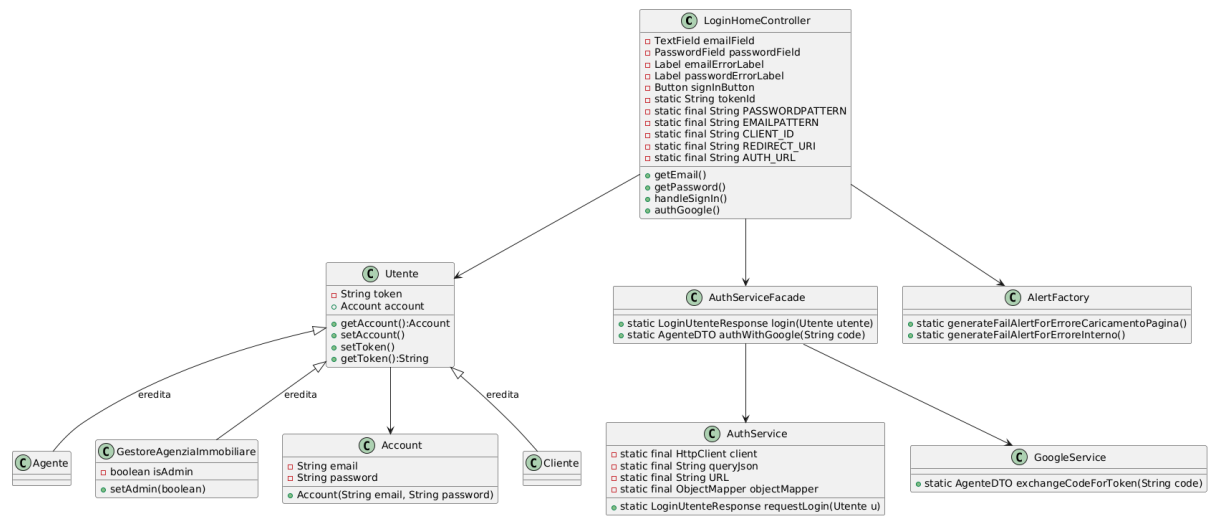
12.5 Crea annuncio



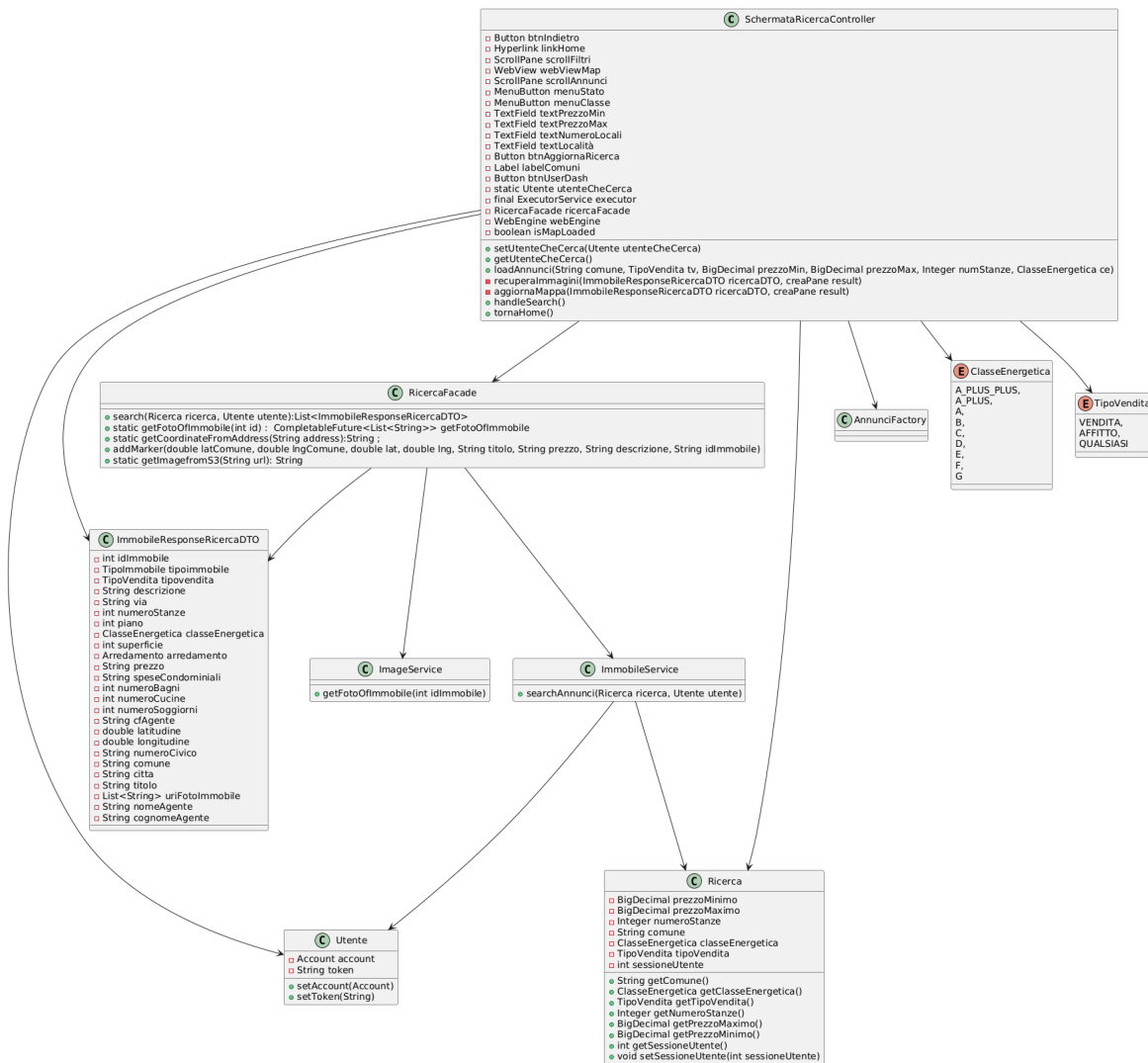
12.6 Modifica profilo



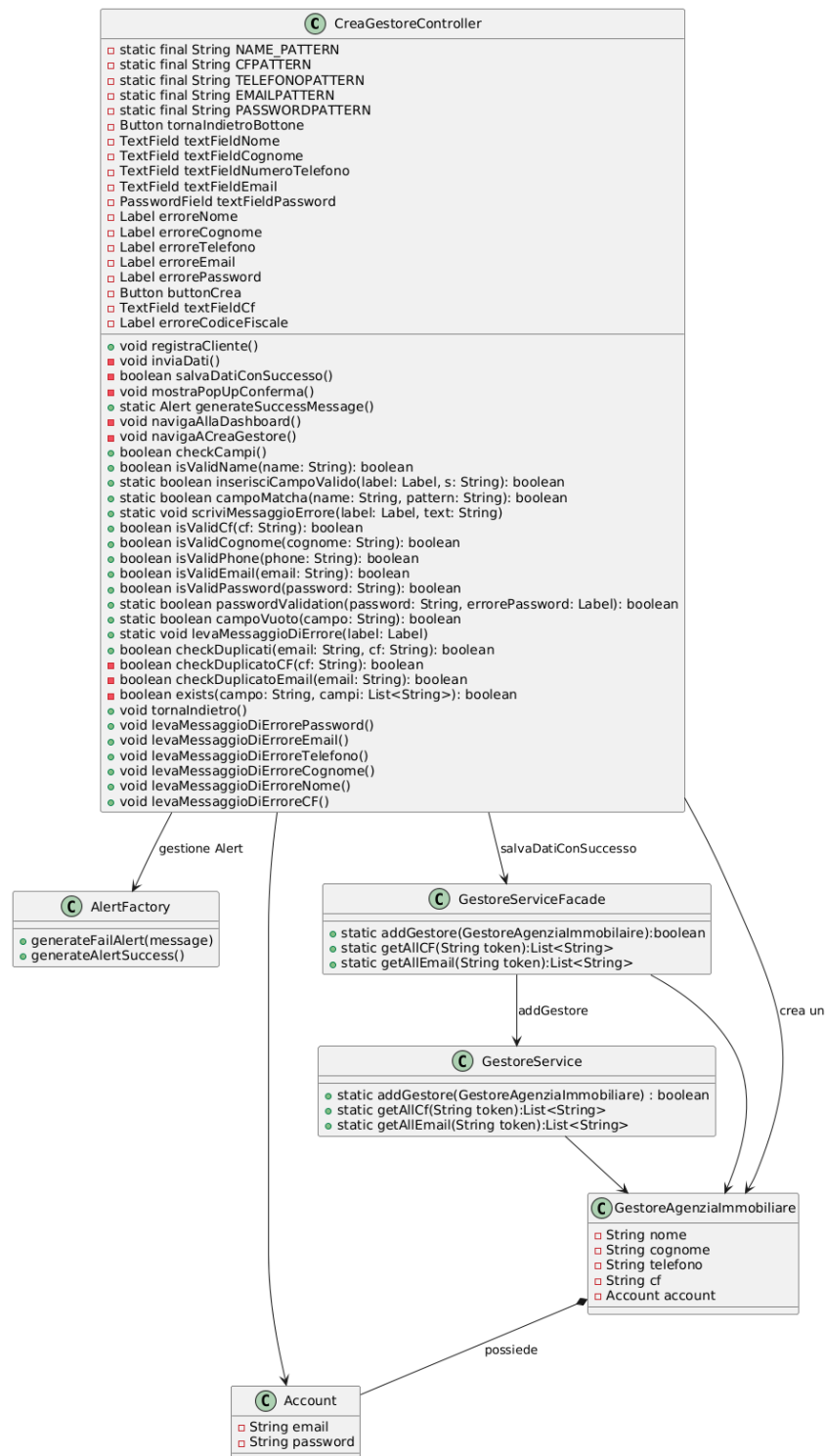
12.7 Autenticazione



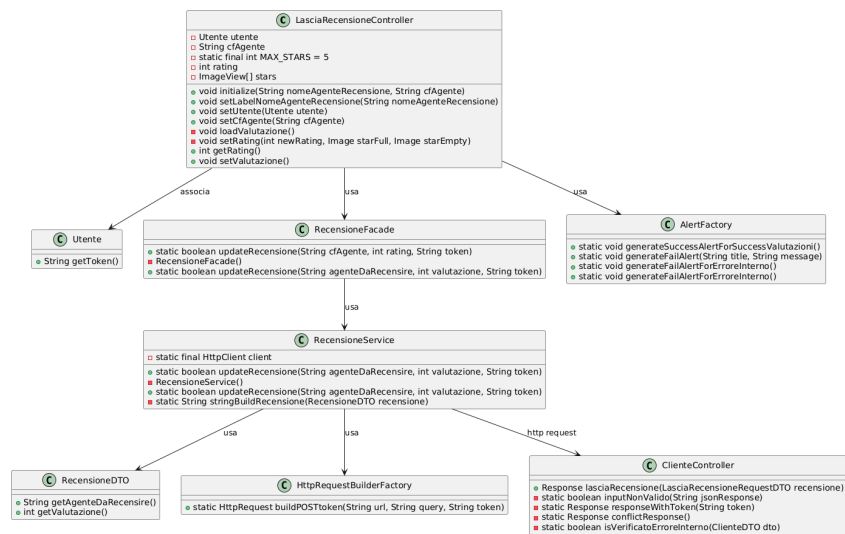
12.8 Ricerca



12.9 Creazione account



12.10 Lascia recensione



13 Diagramma di sequenza di design

I diagrammi di sequenza sono uno strumento essenziale nella modellazione dei sistemi software, in particolare nella fase di analisi e progettazione.

Questi diagrammi sono particolarmente utili per descrivere casi d'uso specifici, il comportamento dei sistemi distribuiti e le interazioni tra componenti software. **Forniscono una visione chiara della logica operativa, facilitando la comunicazione tra sviluppatori, analisti e stakeholder.**

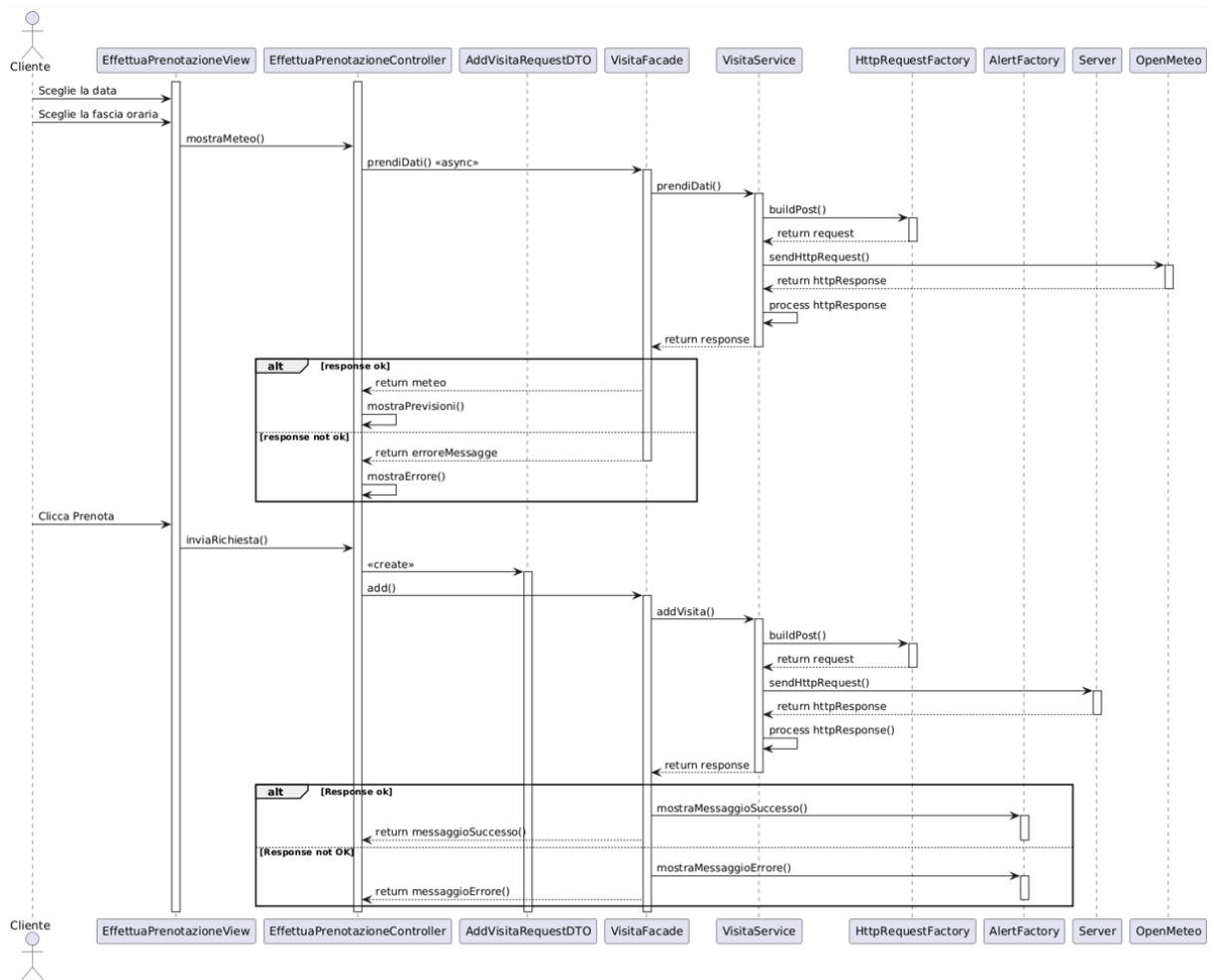
13.1 Caso d'uso : Prenota visita

Il seguente diagramma di sequenza illustra il **processo di effettuazione di una prenotazione da parte del cliente**. L'attore principale, il cliente, seleziona una data tra quelle disponibili e una fascia oraria in cui desidera visitare l'immobile.

A questo punto, **il sistema interagisce con un servizio esterno denominato OpenMeteo** per ottenere le previsioni meteorologiche relative alla data selezionata, le quali vengono successivamente presentate al cliente.

Successivamente, il cliente conferma la prenotazione cliccando sull'apposito pulsante. Il sistema elabora i dati forniti e invia una richiesta al server per la registrazione della prenotazione.

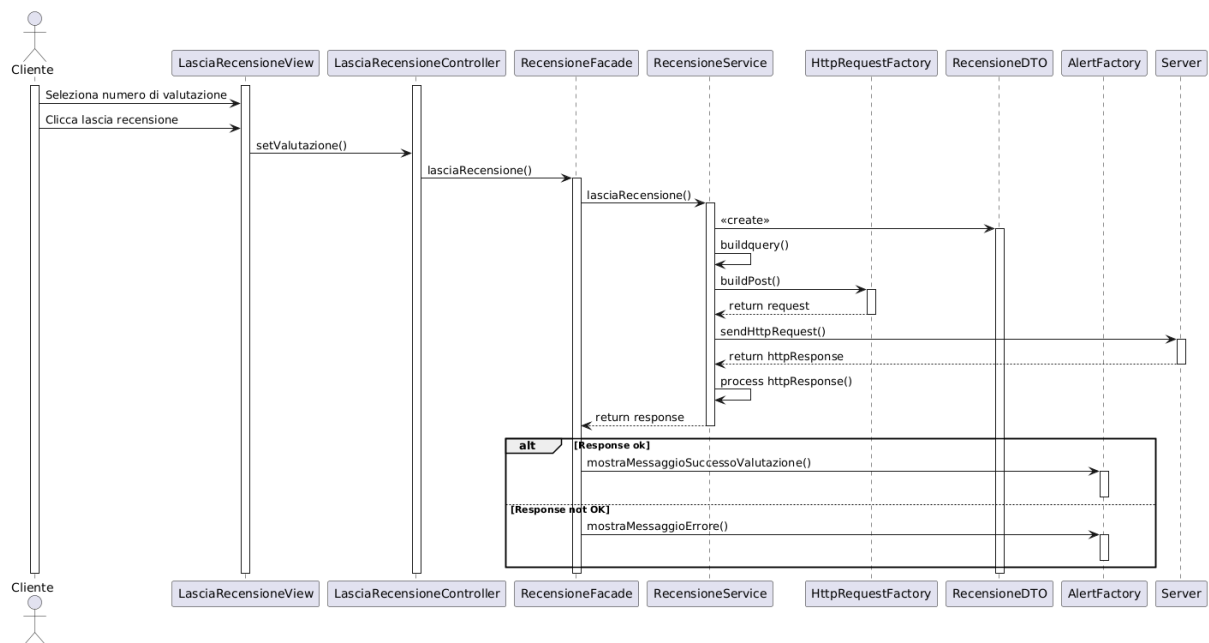
Se la richiesta viene elaborata con successo, il sistema aggiorna lo stato della prenotazione e notifica il cliente tramite un messaggio **di conferma visualizzato nell'interfaccia grafica**. In caso contrario, il sistema interpreta il **messaggio di errore restituito dal server e fornisce al cliente una comunicazione chiara e dettagliata riguardo al problema riscontrato**.



13.2 Caso d'uso: Lascia recensione

Il seguente diagramma di sequenza illustra il processo di **rilascio di una recensione da parte del cliente**. L'attore principale, il cliente, seleziona una valutazione da assegnare all'agente e procede **clickando sul pulsante "Invia recensione"**. Il sistema elabora i dati forniti e invia una richiesta al server per la registrazione della recensione.

Qualora la richiesta venga elaborata con successo, **il sistema notifica il cliente mediante un messaggio di conferma visualizzato nell'interfaccia grafica**. In caso di errore, il sistema interpreta il messaggio di errore restituito dal server e fornisce al cliente un avviso adeguato, descrivendo il problema riscontrato.



Artefatti Software

In questa sezione analizziamo il Dockerfile e il processo di build automatica per configurare e distribuire l'applicazione backend in un ambiente Dockerizzato.

14 DockerFile e file di build Automatica

In questa sezione analizziamo il Dockerfile e il processo di build automatica per configurare e distribuire l'applicazione backend in un ambiente Dockerizzato.

14.1 File di build Automatica

I file di build automatica sono utilizzati per gestire il processo di costruzione di un progetto software in modo automatizzato. Essi definiscono le istruzioni necessarie per compilare, testare, pacchettizzare e distribuire il codice, garantendo coerenza e riproducibilità nel processo di sviluppo.

14.2 DockerFile

```
FROM openjdk:21-jdk

COPY target/ProvaProgettoV2-1.0-SNAPSHOT.jar /usr/app/

WORKDIR /usr/app

ENTRYPOINT ["java", "-jar", "ProvaProgettoV2-1.0-SNAPSHOT.jar"]
```

DockerFile Back-end

Il **Dockerfile** definisce l'ambiente di **build e runtime** necessario per eseguire un'applicazione Java all'interno di un container Docker. In particolare, questa configurazione costruisce un'immagine basata su **OpenJDK 21**, includendo il file `.jar` dell'applicazione e definendo il processo di avvio.

14.2.1 Struttura del Dockerfile

- **Base Image:** Viene utilizzata l'immagine ufficiale `openjdk:21-jdk`, che fornisce un ambiente Java completo.
- **Copia dell'Applicazione:** Il comando `COPY` trasferisce il file `ProvaProgettoV2-1.0-SNAPSHOT.jar` nella directory `/usr/app/` all'interno del container.
- **Impostazione della Directory di Lavoro:** Il comando `WORKDIR /usr/app` definisce la directory di lavoro, semplificando l'esecuzione dei comandi successivi.

- **Avvio dell'Applicazione:** L'ENTRYPOINT specifica il comando da eseguire quando il container viene avviato, utilizzando `java -jar` per eseguire il file `.jar`.

Questa configurazione garantisce che l'applicazione venga eseguita in modo isolato e riproducibile in qualsiasi ambiente Docker.

14.2.2 Docker compose

```
services:
  dieti-estates-app:
    build: .
    ports:
      - "9094:9094"
    stdin_open: true
    tty: true
    depends_on:
      - database
    environment:
      DATABASE_URL: "jdbc:postgresql://database:5432/DietiEstates"
      DATABASE_USER: "neondb_owner"
      DATABASE_PASSWORD: "${DATABASE_PASSWORD}"
      DATABASE_SSL_MODE: "require"

  database:
    image: postgres:15
    restart: always
    environment:
      POSTGRES_DB: DietiEstates
      POSTGRES_USER: neondb_owner
      POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}"
    ports:
      - "5432:5432"
```

Docker Compose Back-end

Il file `docker-compose.yml` definisce due servizi principali: l'applicazione DietiEstates e il database PostgreSQL. Il file configura la creazione e la gestione di un'applicazione Docker, che include le variabili di ambiente necessarie per la connessione al database, nonché le impostazioni di rete e di accesso.

14.2.3 6.1.3 Accesso Interattivo al Container

Le opzioni `stdin_open` e `tty` sono abilitate per consentire l'accesso interattivo al container.

- `stdin_open`: Abilita il flusso di input da tastiera (`stdin`) nel container.
- `tty`: Consente di allocare un terminale per l'accesso interattivo.

14.2.4 Gestione delle Password

Le password, come `DATABASE_PASSWORD` e `POSTGRES_PASSWORD`, sono gestite **tramite variabili d'ambiente definite in un file `.env`**. In questo modo, le credenziali non sono esposte direttamente nel file di configurazione `docker-compose.yml`, garantendo una maggiore sicurezza. Questa modalità di gestione delle password permette di mantenere le informazioni sensibili al sicuro, evitando che vengano visibili nel codice o nel file di configurazione, riducendo il rischio di esposizione accidentale.

15 Strumento di versioning

15.1 Scelte adottate

GitHub è una piattaforma di hosting per il controllo di versione e la collaborazione, costruita attorno al sistema di versioning distribuito Git. Questo strumento è ampiamente utilizzato nella comunità degli sviluppatori per gestire progetti software. Per il versioning di **DietiEstates25** abbiamo scelto GitHub.

Vantaggi di GitHub come strumento di versioning

L'utilizzo di GitHub come strumento di versioning presenta numerosi vantaggi, tra cui:

- **Collaborazione facilitata:** GitHub consente a più sviluppatori di lavorare contemporaneamente sullo stesso progetto senza conflitti. Attraverso le funzionalità di pull request, gli utenti possono proporre modifiche al codice, che possono essere revisionate e integrate nel progetto principale. Questo processo rende la collaborazione più fluida e organizzata.
- **Tracciamento delle modifiche:** Ogni modifica apportata al codice è registrata nella cronologia del repository. Gli sviluppatori possono visualizzare chi ha apportato una modifica, quando e perché, il che aiuta a mantenere una documentazione chiara delle evoluzioni del progetto.
- **Ritorno a versioni precedenti:** GitHub consente di annullare modifiche o di tornare a versioni precedenti del codice. Questo è particolarmente utile quando una nuova funzionalità introduce un bug o quando è necessario ripristinare un comportamento precedente.
- **Backup e sicurezza :** I repository su GitHub sono ospitati nel cloud, fornendo un backup sicuro del codice.

15.2 Repository

 TonyKing1510 Aggiunta implementazione Finale e risolti alcuni bug di Git a2215da · now  82 Commits		
 ImplementazioneFinale/DietiEstates25Versione...	Aggiunta l'implementazione finale del progetto e anche alcu...	5 minutes ago
 ProveCodiceProgetto	Aggiunta versione finale del progetto	15 minutes ago
 Requisiti_Del_Sistema	Aggiunte Faq	4 months ago
 documentazione	Aggiunta l'implementazione finale del progetto e anche alcu...	5 minutes ago
 README.md	Update README.md	5 months ago

Repository usata per DietiEstates25

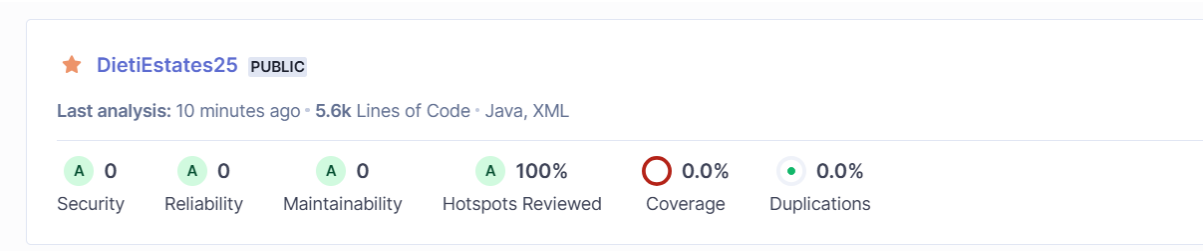
16 Report di qualità del codice

La qualità del codice rappresenta un elemento fondamentale nello sviluppo software. Un codice di alta qualità facilita la comprensione e la manutenzione da parte degli sviluppatori, riduce il numero di bug e vulnerabilità, aumentando la stabilità dell'applicazione. Inoltre, un codice ben strutturato e pulito è essenziale per garantire la scalabilità del progetto. Permette di aggiungere nuove funzionalità senza compromettere le performance o l'affidabilità del sistema.

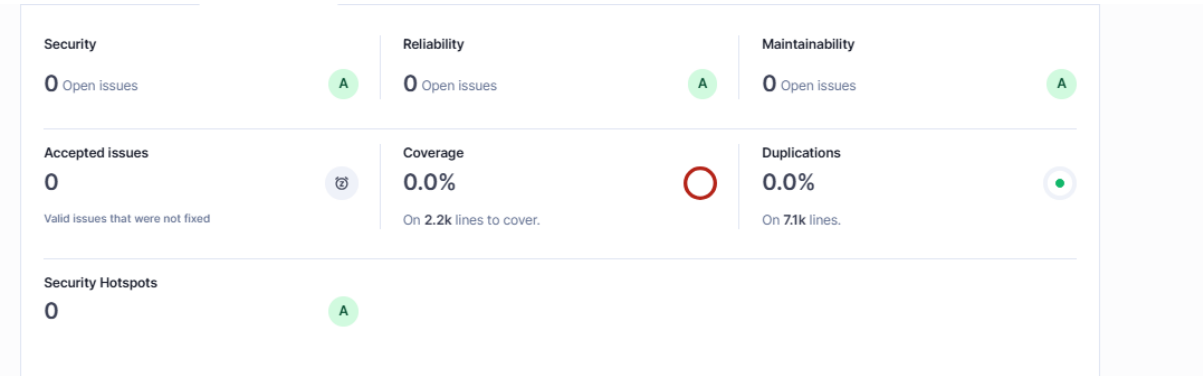
16.1 SonarQube

Per monitorare e migliorare la qualità del codice, è stato impiegato SonarQube. Questo permette di identificare potenziali bug, vulnerabilità e problemi tecnici, fornendo suggerimenti per ottimizzare il codice.

- **Duplicazioni** : Le duplicazioni di codice possono portare a una maggiore complessità e difficoltà nella manutenzione. SonarQube rileva segmenti di codice duplicati e suggerisce di refattorizzarli per migliorare l'efficienza e ridurre il rischio di inconsistenze.
- **Bugs** : Rappresentano errori nel codice che possono causare comportamenti imprevisti o malfunzionamenti dell'applicazione.
- **Vulnerabilità** : Questi indici segnalano potenziali rischi di sicurezza presenti nel codice.
- **Code Smells** : Indicano aree del codice che, pur non essendo errori diretti, possono ridurre la leggibilità, la manutenibilità o la performance dell'applicazione.



Report di SonarQube per DietiEstates25



Report di SonarQube per DietiEstates25

Testing

16.2 Unit Testing

All'interno del progetto, lo Unit Testing è stato implementato seguendo la metodologia **Black Box Testing**. Questo approccio si concentra sulla verifica della correttezza delle funzionalità esposte dai moduli del software senza analizzare la loro implementazione interna. **L'obiettivo è garantire che, dati specifici input, il sistema restituisca gli output attesi, verificando il comportamento complessivo dell'applicazione piuttosto che il codice sorgente sottostante.**

Per l'implementazione dei test unitari è stato utilizzato il framework **JUnit**, che adotta il modello xUnit, come richiesto dal committente. Tale modello è ampiamente riconosciuto per la strutturazione e l'automazione dei test, consentendo un'efficace organizzazione delle verifiche e una gestione strutturata dei casi di test

16.2.1 lasciaRecensione

Il primo metodo che abbiamo scelto di testare è il metodo **Lascia Recensione** del back-end.

Questo metodo accetta due parametri:

- Il **codice fiscale** dell'agente.
- La **valutazione** assegnata dal cliente.

Il metodo restituisce un **booleano** che indica se l'invio della valutazione è andato a buon fine.

Per il codice fiscale dell'agente, è stato adottato il **seguito pattern**:

$\sim [A-Z]\{6\}[0-9]\{2\}[A-Z][0-9]\{2\}[A-Z][0-9]\{3\}[A-Z]\$$

Parametro	Classe di Equivalenza	Validità
Codice Fiscale	CE1: Null	Non valida
	CE2: Stringa vuota	Non valida
	CE3: Stringa che non rispetta il pattern	Non valida
	CE4: Stringa che rispetta il pattern	Valida
Valutazione	CE1: ≤ 0	Non valida
	CE2: > 5	Non valida
	CE3: $1 \leq \text{Valutazione} \leq 5$	Valida

Table 1: Classi di Equivalenza e validità dei parametri per il metodo lascia recensione

Adottiamo il criterio di copertura **R-WECT** (Weak Robust Equivalence Class Testing), che offre un buon livello di copertura riducendo, al contempo, il numero di test case necessari. Questo approccio rappresenta un compromesso efficace tra i criteri **SECT** e **WECT**.

Le classi di equivalenza non valide vengono testate separatamente da quelle valide, mentre ciascuna classe valida viene verificata una sola volta. In totale, il numero di test case **risultante è pari a sei**.

Per il caso di test in questione, abbiamo deciso di adottare l'uso dei **Mock**. **Questa scelta è stata fatta per evitare di interagire con i dati reali degli agenti, garantendo così l'integrità e la sicurezza dei dati sensibili**. In particolare, per implementare i mock, abbiamo scelto di utilizzare il **framework Mockito**, una libreria di test per Java che permette di simulare il comportamento delle classi e degli oggetti in modo controllato durante l'esecuzione dei test.

L'adozione dei mock consente di isolare il codice sotto test, evitando dipendenze da componenti esterni come nel nostro caso inserire la valutazione nel database.

```
class ClienteControllerTest {

    @InjectMocks
    private ClienteController controller;

    @Mock
    private ClienteService clienteService;

    @Mock
    private ClienteDao clienteDao;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks( testClass: this);
    }

    private LasciaRecensioneRequestDTO creaRecensione(String agenteDaRecensire, int valutazione) {
        LasciaRecensioneRequestDTO recensioneDTO = new LasciaRecensioneRequestDTO();
        recensioneDTO.setAgenteDaRecensire(agenteDaRecensire);
        recensioneDTO.setValutazione(valutazione);
        return recensioneDTO;
    }
}
```

Figure 2: Codice JUnit per il metodo Lascia recensione

```

@ParameterizedTest
@CsvSource({
    "null, 1",      // Codice fiscale null
    "", 2",        // Codice fiscale vuoto
    "SVN", 5",     // Codice fiscale non valido
    "SVNGLN03T05H892R", -1", // Valutazione negativa
    "SVNGLN03T05H892R", 6"  // Valutazione superiore a 5
})
void testLasciaRecensione_InputNonValido(String agenteDaRecensire, int valutazione) {
    // Arrange
    LasciaRecensioneRequestDTO recensioneDTO = creaRecensione(agenteDaRecensire.equals("null") ? null : agenteDaRecensire
        , valutazione);
    // Mock dei metodi
    Mockito.when(clienteDao.lasciaRecensione(Mockito.any(LasciaRecensioneRequestDTO.class))).thenReturn(t false);
    Mockito.when(clienteService.lasciaRecensione(Mockito.any(LasciaRecensioneRequestDTO.class))).thenReturn(t false);
    // Act
    Response response = controller.lasciaRecensione(recensioneDTO);
    // Assert
    assertEquals(Response.Status.BAD_REQUEST.getStatusCode(), response.getStatus());
    //Nessun interazione perchè bloccato dai controlli del controller
    verifyNoInteractions(clienteService);
}

```

```

@ParameterizedTest
@CsvSource({
    "SSKTGH03T05H892R", 4", // Codice fiscale valido e valutazione valida
    "LCMGLN03T05H892R", 3" // Altro caso valido
})
void testLasciaRecensione_InputValido(String agenteDaRecensire, int valutazione) {
    // Arrange: Creiamo la richiesta DTO
    LasciaRecensioneRequestDTO recensioneDTO = creaRecensione(agenteDaRecensire.equals("null") ? null : agenteDaRecensire
        , valutazione);

    // Mock dei metodi
    Mockito.when(clienteDao.lasciaRecensione(Mockito.any(LasciaRecensioneRequestDTO.class))).thenReturn(t true);
    Mockito.when(clienteService.lasciaRecensione(Mockito.any(LasciaRecensioneRequestDTO.class))).thenReturn(t true);

    // Act Chiamata al controller
    Response response = controller.lasciaRecensione(recensioneDTO);

    // Assert Verifica che il controller restituisce status code esatto
    assertEquals(Response.Status.OK.getStatusCode(), response.getStatus());

    //Verifico
    verify(clienteService).lasciaRecensione(Mockito.any(LasciaRecensioneRequestDTO.class));
}

```

Figure 3: Codice JUnit per il metodo Lascia recensione

16.2.2 checkVisitaPerCliente

Il secondo metodo che abbiamo scelto di testare è il metodo **checkVisitaPerCliente** del back-end. Questo metodo accetta due parametri:

- L' identificativo dell'immobile.
- L'email del cliente.

Il metodo prende questi due parametri e **controlla che il cliente con quella email abbia già una visita per quell'immobile**. Il metodo restituisce un DTO che da informazioni aggiuntive sull'esito della richiesta.

Per l'email del cliente, è stato adottato il seguente pattern:

$$\^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\$$$

Parametro	Classe di Equivalenza	Validità
Email	CE1: Null	Non valida
	CE2: Stringa vuota	Non valida
	CE3: Stringa che non rispetta il pattern	Non valida
	CE4: Stringa che rispetta il pattern	Valida
Id immobile	CE1: Null	Non valida
	CE2: < 0	Non valida
	CE3: = 0	Valida
	CE4: > 0	Valida

Table 2: Classi di Equivalenza e validità dei parametri per il checkVisitaPerCliente

Adottiamo anche qui il criterio di copertura **R-WECT** (Weak Robust Equivalence Class Testing). Quindi in totale, il numero di test case risultante è pari a **sette**.

```

class VisitaControllerTest {
    private VisitaController controller;

    @BeforeEach
    void setUp() { controller = new VisitaController(); }

    private Response callCheckVisita(Integer idImmobile, String email) {
        return controller.checkVisitaPerCliente(idImmobile, email);
    }

    @ParameterizedTest
    @CsvSource({
        "0, '', // Email vuota
        "1, 'invalidEmail', // Email non valida
        "2, null" // Email nulla
    })
    void testCheckVisita_EmailInvalid(Integer idImmobile, String email) {
        // Act
        Response response = callCheckVisita(idImmobile, email);

        // Assert
        assertEquals(Response.Status.BAD_REQUEST.getStatusCode(), response.getStatus());
    }
}

```

Figure 4: Codice JUnit per il metodo checkVisitaPerCliente

```

@Test
void testCheckVisita_EmailValidIdNull(){
    //Arrange
    String email = "email123@gmail.com";
    //Act
    Response response = callCheckVisita(idImmobile: null, email);
    //Assert
    assertEquals(Response.Status.BAD_REQUEST.getStatusCode(), response.getStatus());
}

@Test
void testCheckVisita_EmailValidIdNegativ(){
    //Arrange
    String email = "email123@gmail.com";
    int idImmobile = -1;
    //Act
    Response response = callCheckVisita(idImmobile, email);
    //Assert
    assertEquals(Response.Status.BAD_REQUEST.getStatusCode(), response.getStatus());
}

```

Figure 5: Codice JUnit per il metodo checkVisitaPerCliente

```

@Test
void testCheckVisita_EmailValidIdZero(){
    //Arrange
    String email = "email123@gmail.com";
    int idImmobile = 0;
    //Act
    Response response = callCheckVisita(idImmobile, email);
    //Assert
    assertEquals(Response.Status.BAD_REQUEST.getStatusCode(), response.getStatus());
}

@Test
void testCheckVisita_EmailValidIdPositive(){
    //Arrange
    String email = "email123@gmail.com";
    int idImmobile = 1;
    //Act
    Response response = callCheckVisita(idImmobile, email);
    //Assert
    assertEquals(Response.Status.OK.getStatusCode(), response.getStatus());
}

```

Figure 6: Codice JUnit per il metodo checkVisitaPerCliente

17 Usabilità sul campo

17.1 Raccolta feedback

Per la valutazione dell'usabilità di DietiEstates25, abbiamo adottato diverse tecniche. Una delle prime tecniche utilizzate durante la fase di progettazione del sistema è **stata la raccolta di feedback dagli utenti finali dell'applicazione**.

In particolare, nella fase iniziale del design, avevamo alcuni dubbi sulla struttura delle pagine di registrazione e creazione degli account. Per questo motivo, abbiamo deciso di sottoporre le diverse proposte agli utenti finali, chiedendo loro un parere.

CI SERVE IL TUO AIUTO!

Quale schermata di registrazione preferisci : scelta 1 o scelta 2 :)

arenellatonynapoli@gmail.com [Cambia account](#)

Non condiviso

* Indica una domanda obbligatoria

Scelta 1

Crea i tuoi agenti su DietiEstates25Pro!

Una volta creati i tuoi agenti possono:

- ✓ Accedere alla loro area riservata
- ✓ Pubblicare annunci
- ✓ Ricevere prenotazioni riguardo gli immobili cartati

Crea i tuoi agenti immobiliari!

1. Inserisci i dati anagrafici dell'agente

Nome

Cognome

Codice fiscale

Professione Telefono

2. Inserisci i dati per il suo account

Email

Password

Figure 7

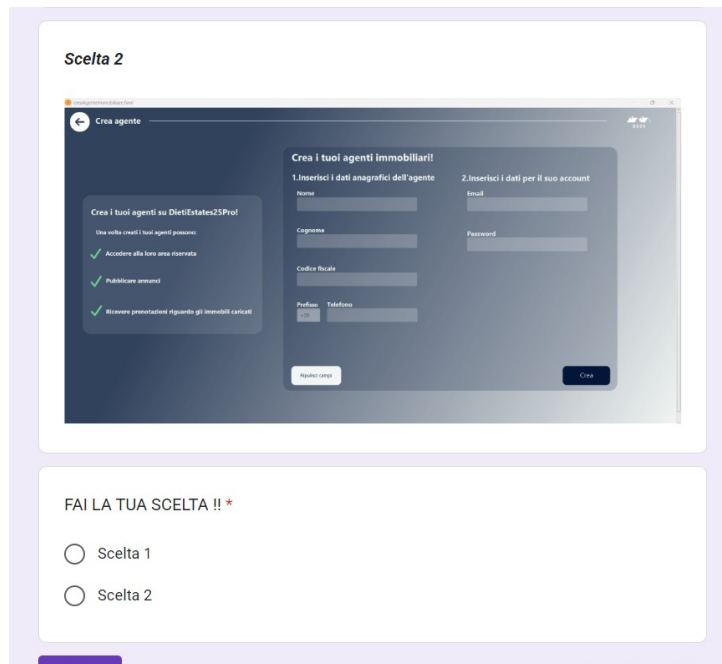


Figure 8

Abbiamo quindi condotto un primo sondaggio, chiedendo agli utenti quale delle schermate preferissero.

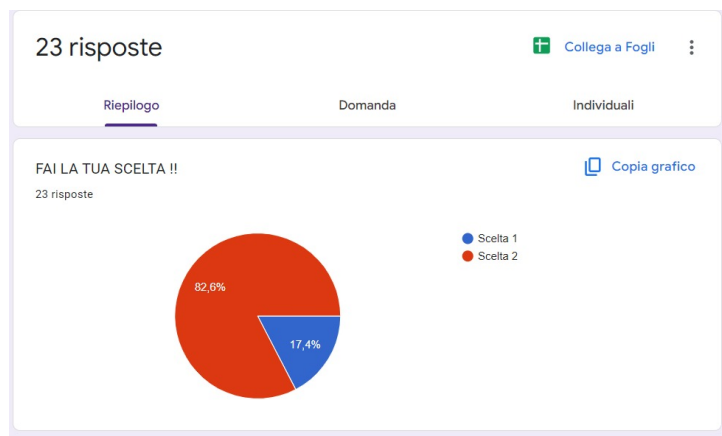


Figure 9

La seconda schermata è risultata la più apprezzata. **Approfondendo le motivazioni della scelta attraverso interviste agli utenti, è emerso che questa versione è stata ritenuta più ordinata e leggibile, grazie a una disposizione degli elementi più chiara e intuitiva.**

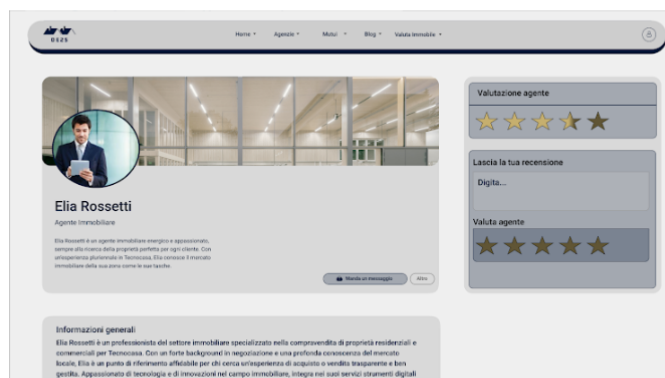
Successivamente abbiamo adottato questo design anche per la pagina di registrazione al sistema.

Abbiamo fatto anche un secondo sondaggio agli utenti finali, in questo caso riguardava il design del processo di lascia recensione. Agli utenti è stato chiesto quale flusso preferissero:



Figure 10

Caso 1: apertura del profilo dell'agente con la parte di valutazione e recensione tutta a destra



Caso 2: apertura del profilo dell'agente con la valutazione vicino al nome ma con il lascia recensione a destra

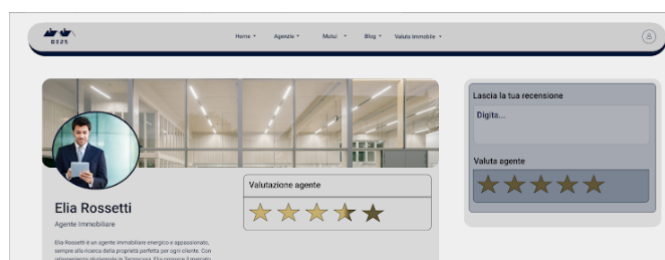
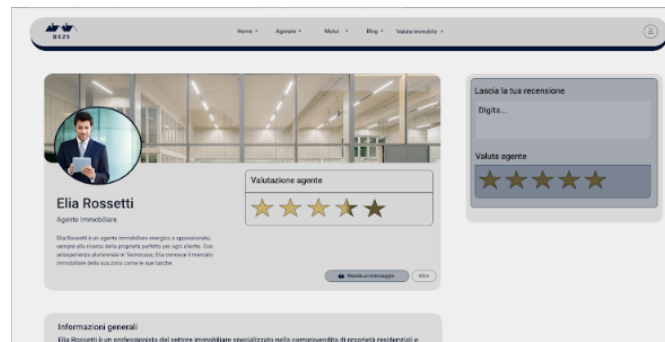


Figure 11

Caso 2: apertura del profilo dell'agente con la valutazione vicino al nome ma con il lascia recensione a destra



Caso 3: pop up che esce sulla pagina appena si clicca il pulsante che ti permette di lasciare la recensione e valutare l'agente

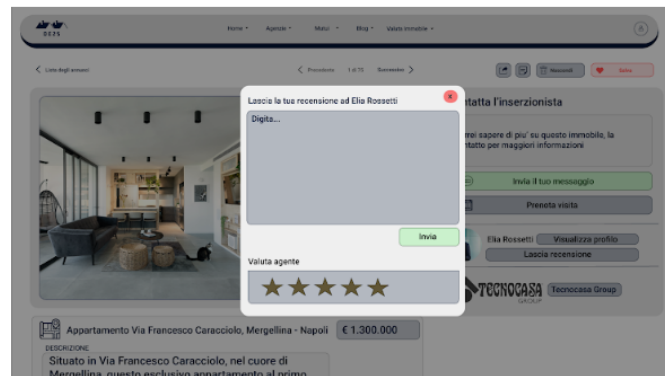


Figure 12

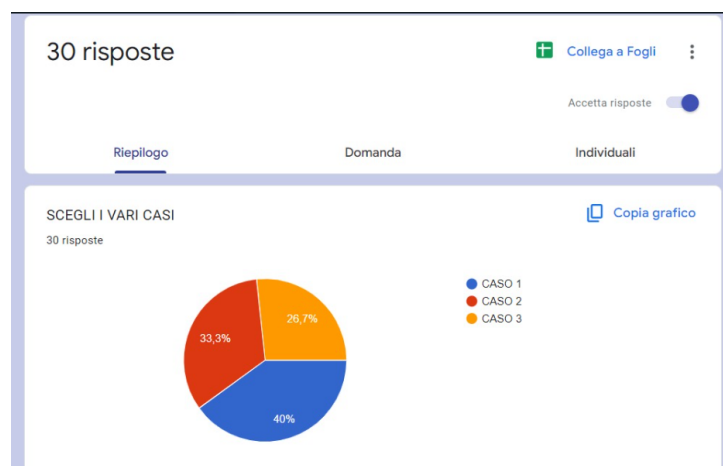


Figure 13

Il processo vincitore è stato il **terzo**(come ci aspettavamo), quindi il design del processo di lascia recensione si è basato su questo feedback degli utenti finali.

18 Valutazione degli esperti

In questa parte ci concentreremo sulla valutazione dell'usabilità di DietiEstates25 da parte di esperti, un metodo in cui esperti di usabilità o di applicazioni specifiche del dominio valutano l'interfaccia utente per identificare problemi, valutare la qualità del design e fornire feedback concreti.

Una delle prime tecniche impiegate per la valutazione da parte degli esperti è stata la valutazione euristica, basata sulle **otto regole d'oro di Shneiderman**.

1. **Sforzarsi per la coerenza** Utilizzare terminologie, layout, colori e comportamenti coerenti in tutta l'interfaccia.
2. **Abilitare scorciatoie per gli utenti esperti** Offrire scorciatoie da tastiera, macro e menu personalizzabili per velocizzare le operazioni.
3. **Fornire un feedback informativo** Ogni azione dell'utente deve generare un feedback chiaro e immediato sullo stato del sistema.
4. **Progettare dialoghi per chiudere un'azione** Strutturare i flussi di lavoro in fasi logiche con chiusure chiare, come la conferma dopo l'invio di un modulo.
5. **Prevenire gli errori** Progettare l'interfaccia per minimizzare gli errori, offrendo conferme, validazioni e annullamenti delle operazioni.
6. **Consentire annullamenti semplici delle azioni** Gli utenti dovrebbero poter annullare o ripetere operazioni senza conseguenze gravi.
7. **Favorire il controllo da parte dell'utente** L'utente deve sentirsi al comando e non costretto da interazioni rigide o automatizzate.
8. **Ridurre il carico cognitivo** Minimizzare la complessità e il sovraccarico di informazioni, visualizzando solo ciò che è necessario al momento giusto.

Facendo un'analisi approfondita della nostre interfacce utente, siamo giunti alla seguente conclusione:

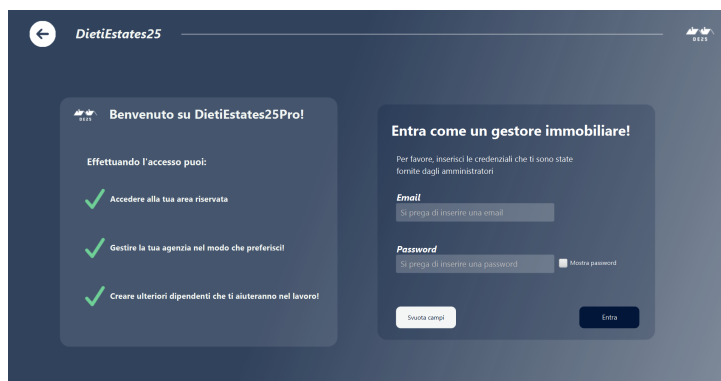


Figure 14

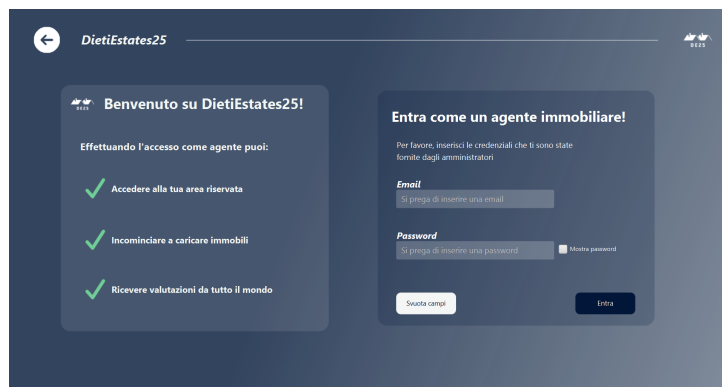


Figure 15

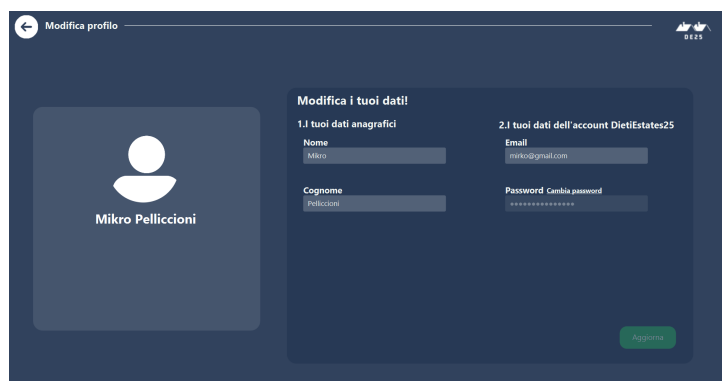


Figure 16

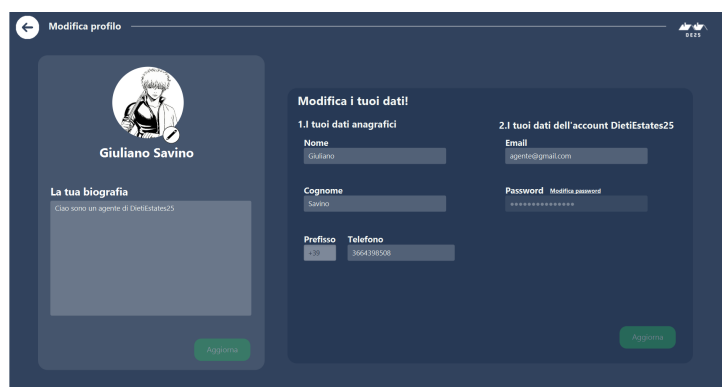


Figure 17

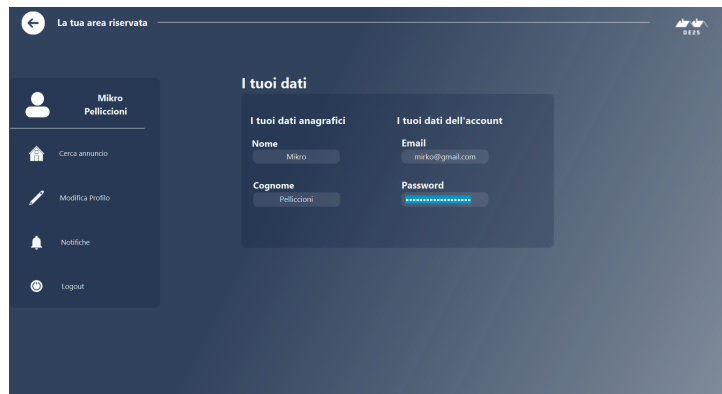


Figure 18

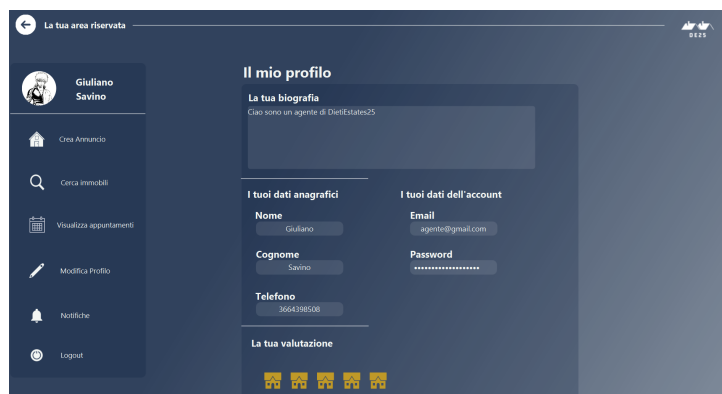


Figure 19

Dall'analisi di queste interfacce utente, riteniamo che il criterio di **coerenza** sia stato rispettato. In particolare:

- I **vantaggi dell'accesso** sono sempre elencati sulla **sinistra**, mentre le informazioni principali si trovano sulla **destra** della pagina.
- L'area riservata presenta un **layout e comportamenti coerenti per tutti i tipi di utenti che utilizzano il sistema**.
- La sezione di modifica del profilo è **uniforme per tutti gli utenti del sistema, con eventuali variazioni dovute a specifiche funzionalità richieste dal committente**.
- Il pulsante che indica l'azione secondaria è **sempre bianco** ed è sempre chiamato *svuota campi*
- Il logo dell'applicazione è sempre in alto a sinistra.

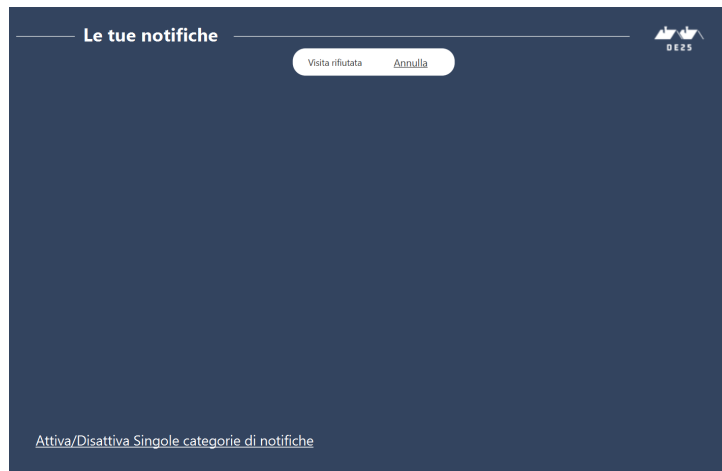


Figure 20

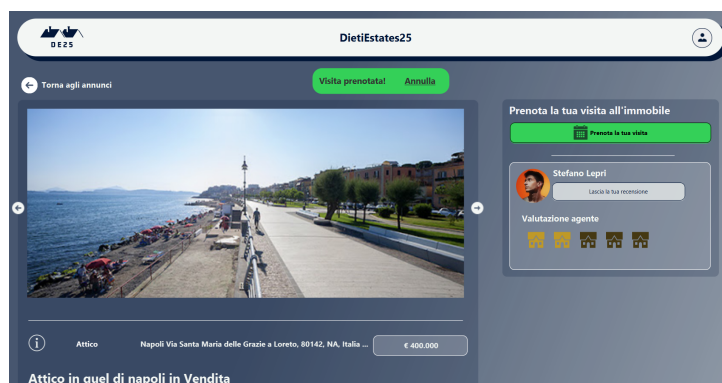


Figure 21

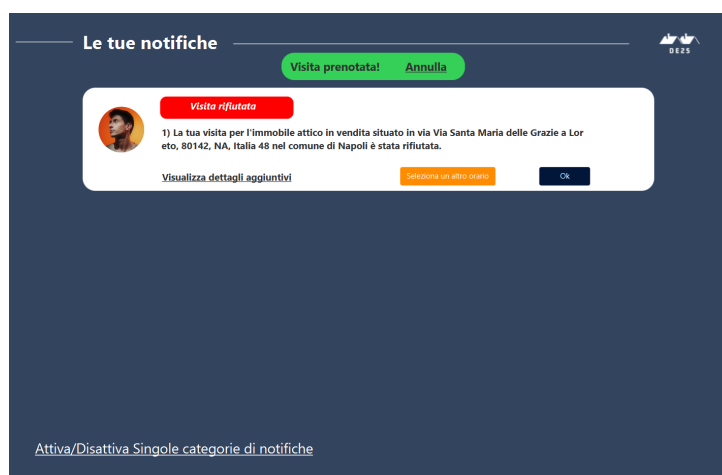


Figure 22

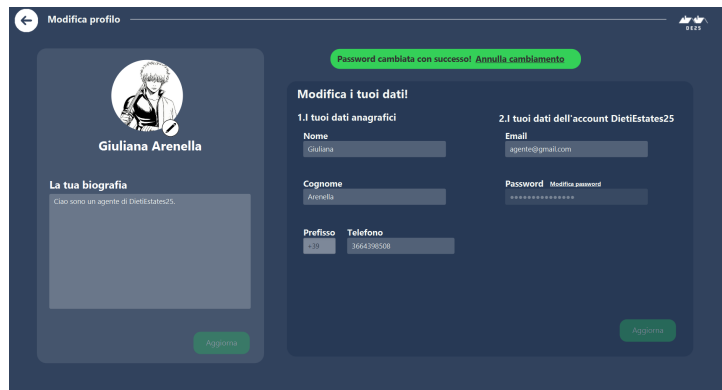


Figure 23

Ogni principale azione dell'utente genera un feedback chiaro e immediato sulla stato del sistema. **Facendo dei test di usabilità** nelle fasi finali del sistema ci siamo resi conto che nella sezione modifica profilo e nel lascito della recensione **non davamo feedback all'utente quindi abbiamo deciso di aggiungerli.**

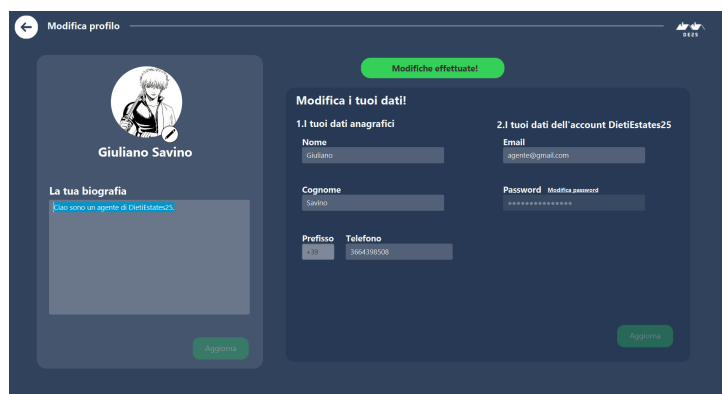


Figure 24

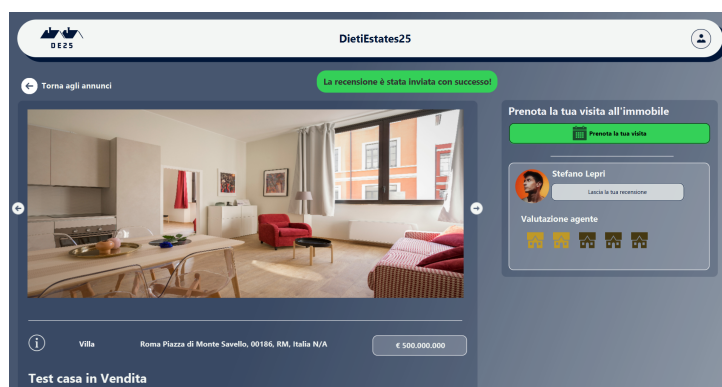


Figure 25

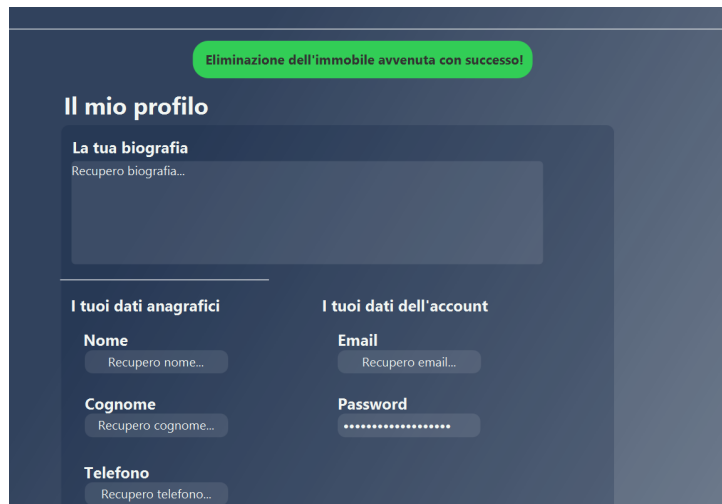


Figure 26

18.1 Fornire un feedback informativo

Ogni azione principale del sistema, **come prenotare una visita, lasciare una recensione, disattivare categorie di notifiche, caricare un immobile, creare nuovi account, modificare il proprio profilo o cancellare un immobile**, fornisce un feedback chiaro e preciso. Questo avviene attraverso pop-up, pannelli informativi che appaiono nella parte superiore della pagina o, in alcuni casi, entrambi i metodi.

18.2 Prevenire gli errori

In qualsiasi schermata, abbiamo cercato di prevenire gli errori che l'utente potrebbe commettere, in particolare:

- Durante la **creazione dell'annuncio**, abbiamo adottato misure per ridurre la possibilità di errore da parte dell'utente. Abbiamo inserito, ove possibile, *radio button* e *menu button* per facilitare la comprensione dell'input richiesto dal sistema. Inoltre, è stata implementata una funzionalità che consente un inserimento semplificato dei prezzi.
- Nella pagina di modifica del profilo, abbiamo previsto meccanismi per evitare errori da parte dell'utente, **come la possibilità di cliccare senza aver effettivamente modificato i dati iniziali**. Inoltre, i vari campi di testo sono stati configurati per impedire l'inserimento di caratteri non validi, come numeri in campi riservati a lettere e viceversa.
- Nel processo di **richiesta di una visita**, abbiamo evitato errori legati all'**inserimento manuale delle date**, adottando un formato in cui l'utente può selezionare la data desiderata tramite un apposito pulsante, senza necessità di digitazione.
- Per la pubblicazione delle recensioni, **abbiamo evitato che l'utente inserisse manualmente una valutazione numerica**, introducendo un sistema di selezione tramite *rating* a stelle, riducendo così il rischio di errore.

- In alcune pagine, invece, abbiamo privilegiato un maggiore controllo da parte dell'utente, **permettendogli di cliccare sempre sul pulsante principale senza che fosse bloccato da input errati**. Questo approccio è stato adottato, ad esempio, nelle pagine di autenticazione e registrazione, al fine di garantire una maggiore libertà d'azione.

Durante la conduzione dei test con gli utenti, **non è emerso alcun caso di errore nei vari form**. Tutti i form sono stati apprezzati e ritenuti intuitivi, risultando efficaci nella prevenzione degli errori.

18.3 Consentire annullamenti semplici delle azioni

Le principali azioni eseguibili dall'utente dispongono di un meccanismo di annullamento semplice e immediato, come la **prenotazione di una visita**, la **modifica della password**, **l'accettazione di una visita** e **l'eliminazione di un annuncio**.

18.4 Ridurre il carico cognitivo

Durante la definizione del sondaggio finale e l'analisi dei feedback dei clienti, **non è emersa alcuna criticità riguardante il carico informativo presentato agli utenti**. Tutti gli utenti hanno apprezzato la quantità di informazioni fornite, ritenendola adeguata per svolgere i compiti richiesti.

18.5 Test con utenti reali

18.5.1 Partecipanti

Il primo passo fondamentale per condurre un test di usabilità è stato il **reclutamento dei partecipanti**, un processo in cui ci siamo concentrati sulla selezione di utenti **diversificati**, tenendo conto di vari fattori tra cui l'età, l'esperienza tecnologica e il background specifico. Questo approccio ha permesso di raccogliere una vasta gamma di feedback, che sono essenziali per ottenere una valutazione completa e rappresentativa del sistema testato. La diversificazione dei partecipanti è cruciale, poiché ogni utente porta con sé una prospettiva unica che può rivelarsi fondamentale per identificare aree di miglioramento e potenziali problematiche nell'interazione con l'interfaccia.

I partecipanti selezionati per il test sono stati:

- **Antonio: Un giovane di 18 anni, estremamente a contatto con la tecnologia** e con una notevole familiarità nell'utilizzo di dispositivi e applicazioni digitali. Antonio è in grado di affrontare l'interfaccia con un approccio rapido e intuitivo. Tuttavia, la sua **esperienza nel settore immobiliare è limitata**, il che rappresenta un'ulteriore variabile da considerare nei test di usabilità.
- **Matteo: Un ragazzo di 13 anni**, noto per la sua velocità nell'apprendere e nell'utilizzare nuovi strumenti tecnologici. Pur avendo una notevole **predisposizione all'uso della tecnologia**, Matteo **non ha esperienze precedenti nel campo immobiliare**, né familiarità con piattaforme

simili. La sua partecipazione al test è stata fondamentale per comprendere come utenti giovani, ma non esperti nel dominio specifico, interagiscono con un sistema che presenta un certo grado di complessità.

- **Adele: Una madre di 49 anni con un'esperienza tecnologica molto limitata.** Sebbene Adele possieda una conoscenza modesta del settore immobiliare, la sua familiarità con l'uso delle tecnologie è praticamente assente. Questa partecipante è stata selezionata per capire come gli utenti con esperienza tecnologica scarsa ma conoscenza del dominio reagiscono a un sistema che potrebbe apparire complesso per chi non ha dimestichezza con l'uso di applicazioni.
- **Fortuna: Una ragazza di 21 anni che, sebbene possieda una buona conoscenza delle tecnologie digitali, ha una limitata comprensione del dominio immobiliare.** Fortuna rappresenta una categoria di utenti che hanno competenze informatiche ma sono relativamente nuovi nel contesto di ricerca di immobili, e la sua esperienza è stata fondamentale per osservare come la familiarità con la tecnologia influisca sull'interazione con l'interfaccia.
- **Sasha: Un agente immobiliare di 55 anni, con un'esperienza consolidata nel settore,** operante in una nota agenzia immobiliare del nostro territorio. Sasha ha una profonda conoscenza del dominio, ma un'esperienza più limitata con le tecnologie digitali avanzate. La sua partecipazione al test ha permesso di esaminare come un professionista del settore immobiliare si interfaccia con il nostro sistema e come gestisce le funzionalità a lui dedicate. I suoi feedback non solo ci hanno fornito indicazioni cruciali sull'usabilità dell'interfaccia, ma anche preziosi suggerimenti su come ottimizzare le funzionalità specifiche del dominio.

18.5.2 Funzionalità testate

Le attività che abbiamo scelto di sottoporre agli utenti sono attività **realistiche** e con un **fine specifico**.

- Registrazione account
- Prenotare visita di un immobile
- Lasciare una recensione
- Caricare immobile
- Modificare il proprio profilo
- Creazione agente
- Accettare richiesta di visita

18.5.3 Metriche considerate

La metrica principale su cui ci siamo basati è il **task completion time**. In particolare, abbiamo definito dei tempi di riferimento per ogni attività e, in base al tempo impiegato da ciascun utente per completarla, gli è stato assegnato un punteggio che indicava il livello di successo.

Abbiamo stabilito che:

- Se l'utente completava il task entro un tempo considerato accettabile, esso veniva classificato come **Superato**.
- Se il tempo impiegato superava leggermente la soglia, ma restava comunque entro limiti accettabili, il task era considerato **Parzialmente Superato**.
- Se il tempo richiesto era eccessivo, il task veniva classificato come **Fallimento**.

Attività	Superato (S)	Superato Parzialmente (SP)	Fallito (F)
Registrazione + Login	30-40 sec	1-2 min	>3 min
Prenotare una visita e lasciare una recensione	1-2 min	2-4 min	>5 min
Caricare un immobile	2-3 min	4-5 min	>6 min
Accettare una richiesta di visita	30-60 sec	1-2 min	>3 min
Login + Creare un agente o gestore	1-3 min	3-5 min	>5 min
Modificare il profilo	30-60 sec	1-2 min	>3 min

Table 3: Metriche di valutazione del tempo per le attività

Per migliorare l'usabilità del prodotto finale, oltre a considerare metriche quantitative come riferimento per ottimizzare l'interfaccia, accogliamo **con interesse qualsiasi feedback, segnalazione di criticità e valutazione del livello di soddisfazione dell'utente**. Ogni suggerimento sarà attentamente esaminato al fine di apportare miglioramenti significativi all'esperienza utente. Quindi come metriche di riferimento per migliorare il nostro prodotto useremo sia metriche **Quantitative** che **Qualitative**.

In particolare, nelle tabelle seguenti sono riportati le funzionalità testate e per ogni utente viene indicato :

- S= **Successo**(ovvero la task è stata completata velocemente senza intoppi)
- SP = **Successo parziale**(ovvero è stato necessario un po' di tempo in più e piccoli suggerimenti)
- F= **Fallimento**(ovvero la task non è stata completata nonostante i piccoli suggerimenti)

18.5.4 Pilota il test

Il test viene condotto su **un partecipante alla volta**, al fine di **garantire un ambiente più tranquillo e consentire una gestione ottimale delle domande rivolte ai singoli utenti**. Oltre ad assegnare loro il compito principale, è stato richiesto anche di fornire un feedback sulla loro esperienza con l'interfaccia utente e sulla percezione e comprensibilità degli elementi che la compongono. Questi sono i risultati avuti dai test di usabilità:

Nome	Reg.+Login	Prenota+Rec.	Carica Imm.	Accetta Visita	Login Crea account	Mod. Profilo	Punti Totali	% Sup.
Antonio	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	600/600	100%
Matteo	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	600/600	100%
Luca	✓ S (100)	✓ S (100)	✓ S (100)	P (50)	P (50)	✓ S (100)	500/600	84%
Fortuna	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	✓ S (100)	600/600	100%
Adele	P (50)	✓ S (100)	P (50)	✓ S (100)	✓ S (100)	✓ S (100)	500/600	84%

Table 4: Tabella dei punteggi e delle percentuali di superamento

Se l'utente completa il task con successo (S), **riceve 100 punti**. Se lo completa solo parzialmente (P), **ottiene 50 punti**. In caso di fallimento (F) , **non riceve alcun punto**. Nella tabella, i task completati parzialmente sono evidenziati in arancione.

18.5.5 Risoluzione errori

Dalle valutazioni effettuate, sono state riscontrate alcune criticità che hanno comportato una rielaborazione di alcuni aspetti dell'interfaccia.

Segnalazione riscontrata	Risoluzione
L'utente, quando caricava un immobile, voleva un messaggio di conferma prima di procedere con il caricamento.	È stato aggiunto un messaggio di conferma prima di caricare un immobile.
L'utente, quando effettuava il logout, voleva un messaggio di conferma prima di uscire.	È stato aggiunto un messaggio di conferma prima del logout.
Abbiamo riscontrato una problematica nei feedback della pagina di modifica profilo: avevamo dato per scontato che modificando i dati nei campi di testo, l'utente si accorgesse immediatamente del cambiamento. Tuttavia, per alcuni utenti questo non era chiaro.	Abbiamo aggiunto feedback positivi espliciti nella pagina di modifica del profilo.
Alcuni utenti hanno segnalato difficoltà dovute alla mancanza di una funzione di autocompletamento.	È stato aggiunto l'autocompletamento nei campi appropriati.

Table 5: Segnalazioni riscontrate e relative risoluzioni

18.5.6 Definizione Survey e Discussione

Per concludere i test di usabilità, abbiamo predisposto un **sondaggio da sottoporre agli utenti al termine della loro esperienza con l'applicativo**. In questo sondaggio, abbiamo innanzitutto richiesto agli utenti alcune **informazioni generali**, inclusa la loro **familiarità con la tecnologia**. Successivamente, abbiamo indagato la loro esperienza d'uso dell'applicativo, chiedendo se le scritte fossero leggibili, se le funzionalità fossero facilmente individuabili e se avessero riscontrato particolari difficoltà nell'utilizzo di alcune funzioni. Infine, abbiamo chiesto se ritenessero opportuna l'implementazione di nuove funzionalità. E questi sono stati i risultati:

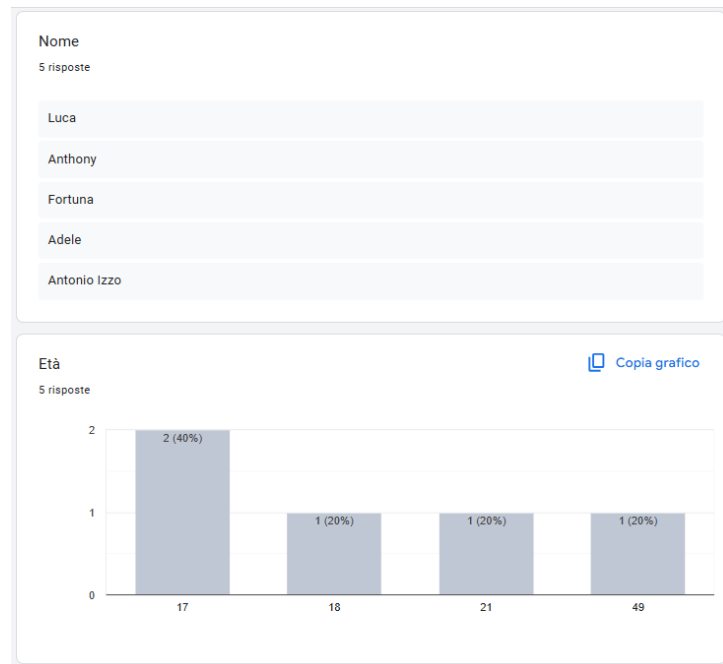


Figure 27

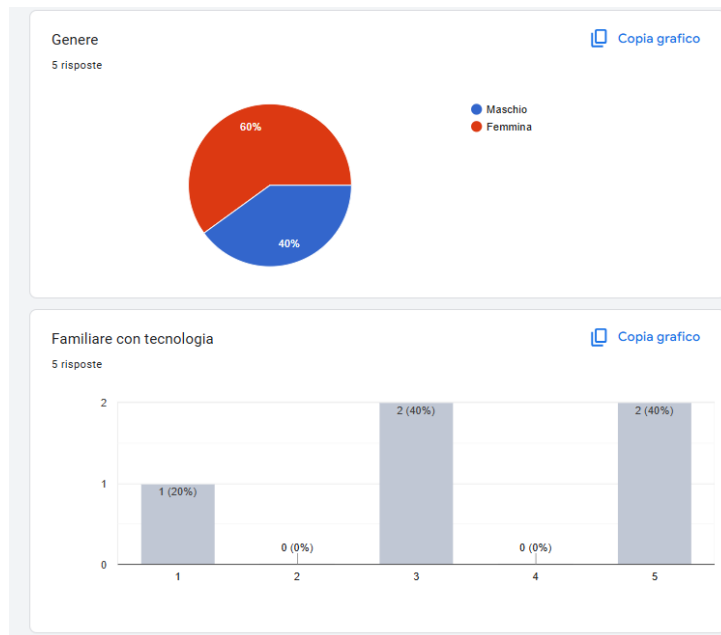


Figure 28



Figure 29

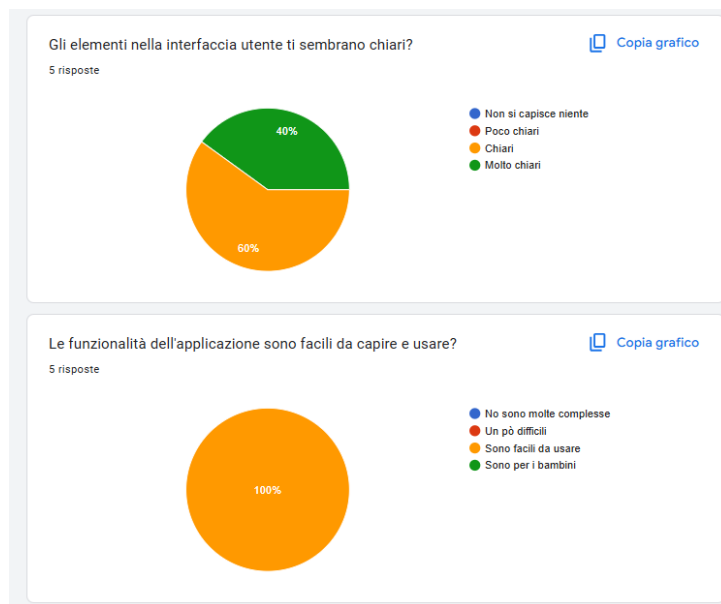


Figure 30

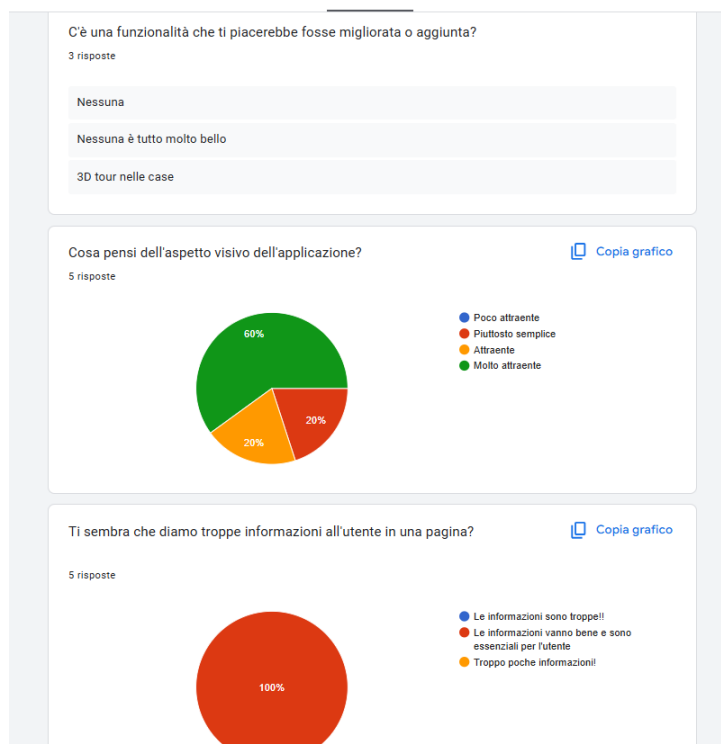


Figure 31



Figure 32

Al termine di questi test, abbiamo constatato che la schermata proposta non presentava **problematiche in termini di semplicità** ed è risultata di facile comprensione. Inoltre, nessun utente ha richiesto il **nostro supporto durante l'utilizzo**. Gli utenti con maggiore esperienza tecnologica hanno immediatamente compreso e padroneggiato l'applicativo.

Il **contrasto dei colori dell'interfaccia** è risultato adeguato e leggibile per tutti gli utenti, inclusi quelli di età avanzata. Anche le **funzionalità più complesse**, come la creazione di un annuncio, non hanno rappresentato un ostacolo significativo per gli utilizzatori.